UNiCON
Unmanned & Intelligent systems CONtrol LAB

# Camera sensor

## You Only Look Once (YOLO)

Department of Artificial Intelligence
Hyeongjun Park
2026.11.06

# Contents


1. Overview of YOLO algorithm

   1) YOLO 개요 및 평가 지표

2. YOLO v12

   1) YOLO v12 학습을 위한 Dataset 구축

   2) YOLO v12 기반 객체 인지 모델 학습

   3) YOLO v12 기반 객체 인지 모델 검증

3. Local 기반 YOLO 실습

   1) 로컬 추론 및 웹캠 실시간 탐지

# 1. Overview of YOLO algorithm

UNICON
Unmanned & Intelligent systems CONtrol LAB

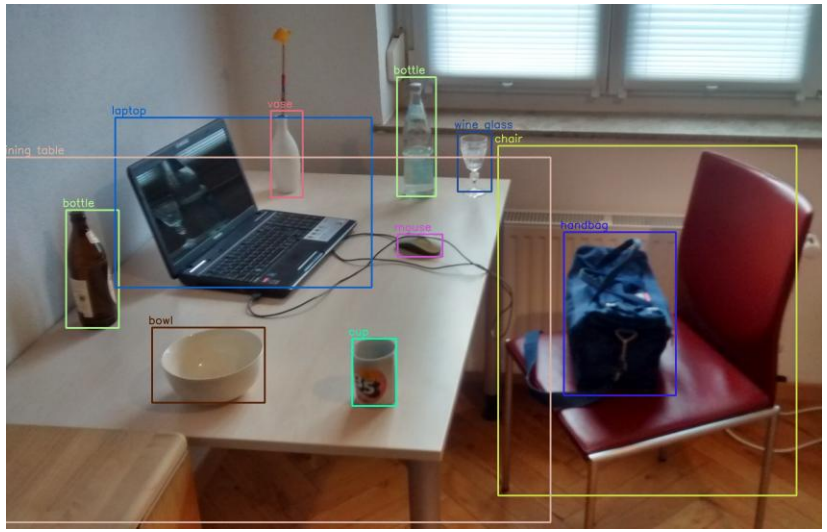
## 1) YOLO(You Only Look Once) 알고리즘의 개요

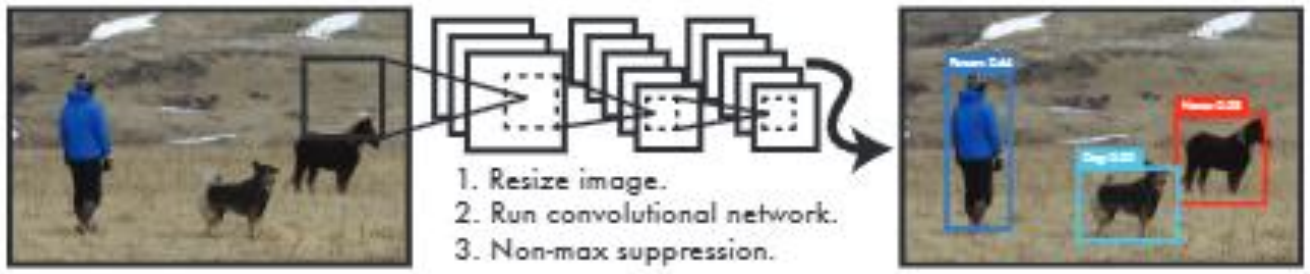
- **YOLO 알고리즘이란?**

- **YOLO(You Only Look Once)**는 합성곱 신경망(CNN)을 기반으로 한 **단일 단계(single-stage) 객체 탐지 알고리즘**

- 기존 2-Stage 방식(R-CNN)과 달리, 이미지 전체를 한 번에 입력으로 받아 **객체의 위치(bounding box)와 클래스(class probability)를 동시에 예측**

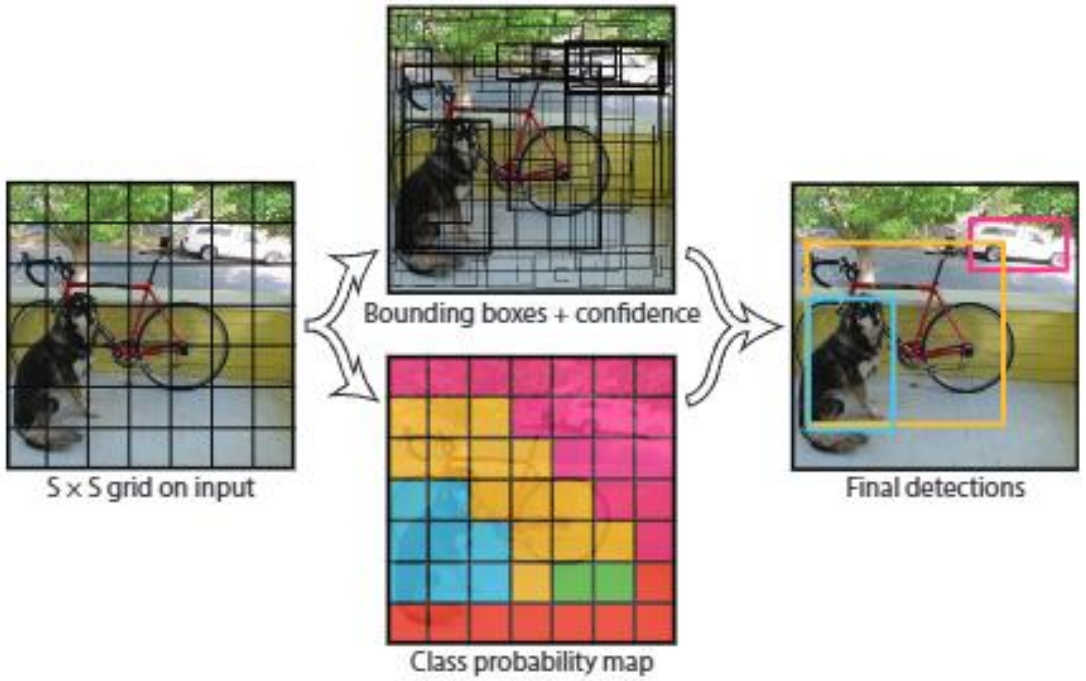
- **proposal + classification 문제**가 아니라 **end-to-end 단일 회귀 문제(regression problem)**로 모델링



UNiCON
Unmanned & Intelligent systems CONtrol LAB

- **YOLOv1**




- **핵심 개념**

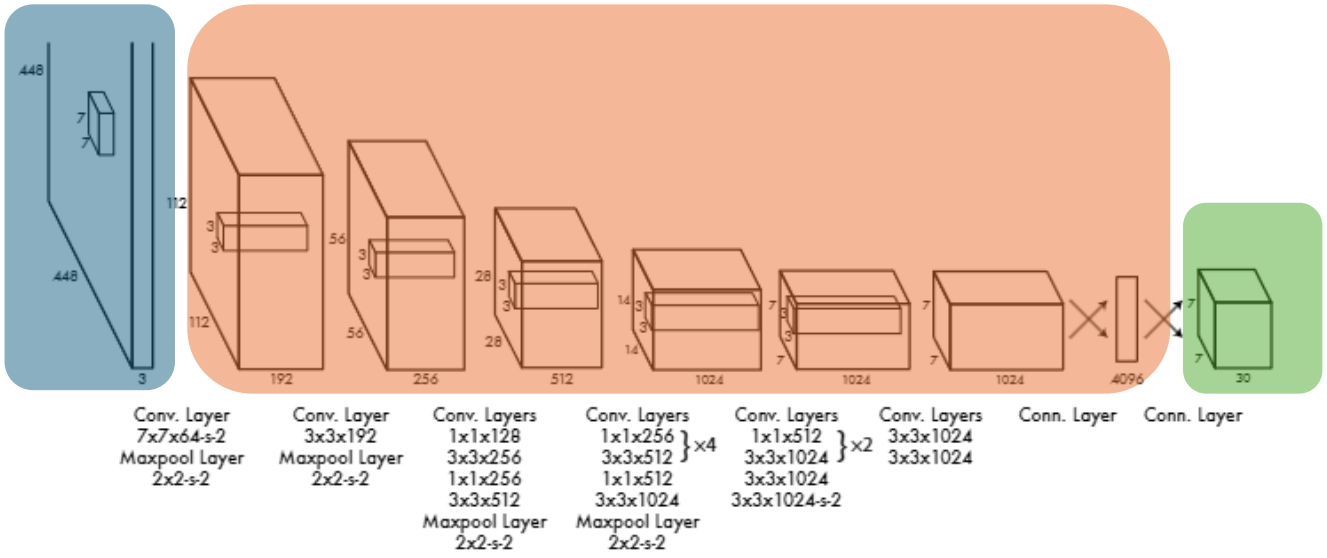
  - **이미지 분할**
    **->** 입력 이미지를 **S x S grid**로 나눔
    -> 각 cell이 객체 1개를 담당

  - **각 Grid Cell의 예측**
    **-> Bounding Box (x,y,w,h)**
    -> **Objectness (Confidence)** : 객체 존재 확률
    -> **Class Probability** : 클래스 확률

  - **Detection 생성**
    **->**모든 cell의 예측을 통합
    ->**Confidence threshold** 적용
    ->**NMS(Non-Maximum Suppression)**로 중복 제거

UNICON
Unmanned & Intelligent systems CONtrol LAB

- **YOLOv1 Network Architecture**




<YoLov1 Architecture>

**(1)Input Image**

- 입력 크기: **448 × 448 × 3**
- 전체 이미지를 한 번에 입력

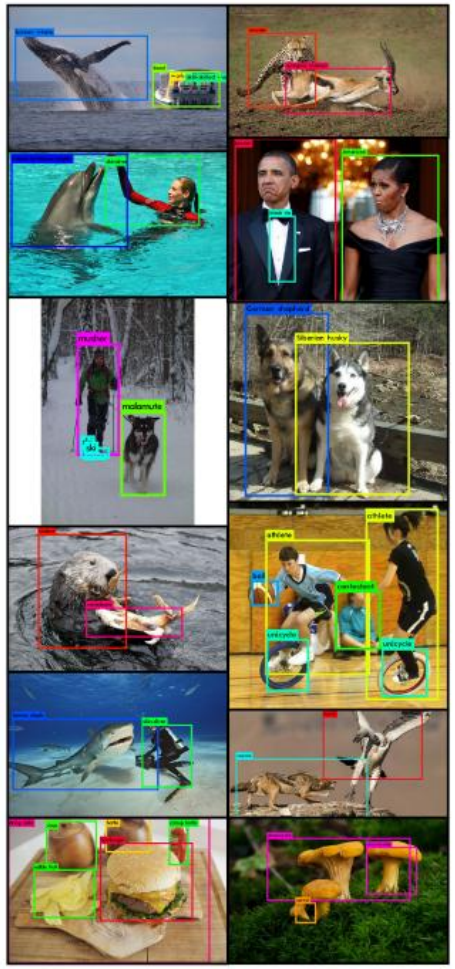
**(2) Feature Extraction**
**Convolutional Layers**

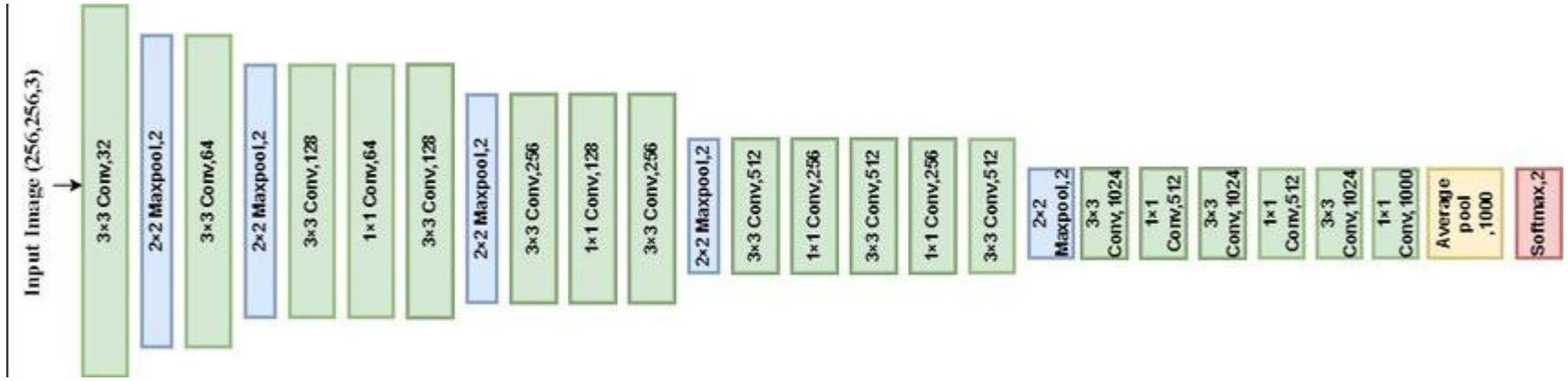
- 합성곱 계층과 max-pooling을 반복
- 이미지의 공간적 특징 추출
- 채널 수 증가, 해상도 감소

**(3) Detection Head**
**Fully Connected Layers**

- 추출된 feature를 기반으로
- **7 × 7 × 30** 형태의 출력 생성
- 각 grid cell이 bounding box와 class 확률 예측

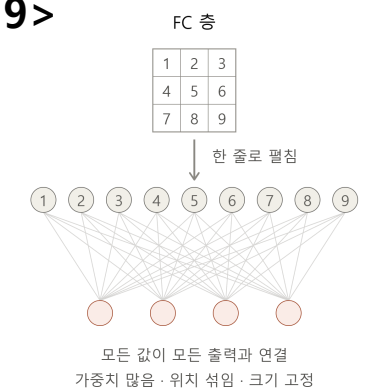
- **YOLOv2 – 현대 YOLO의 기반**

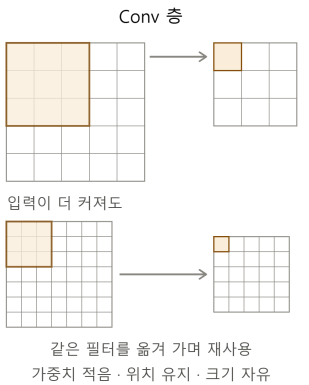





<DarkNet-19>

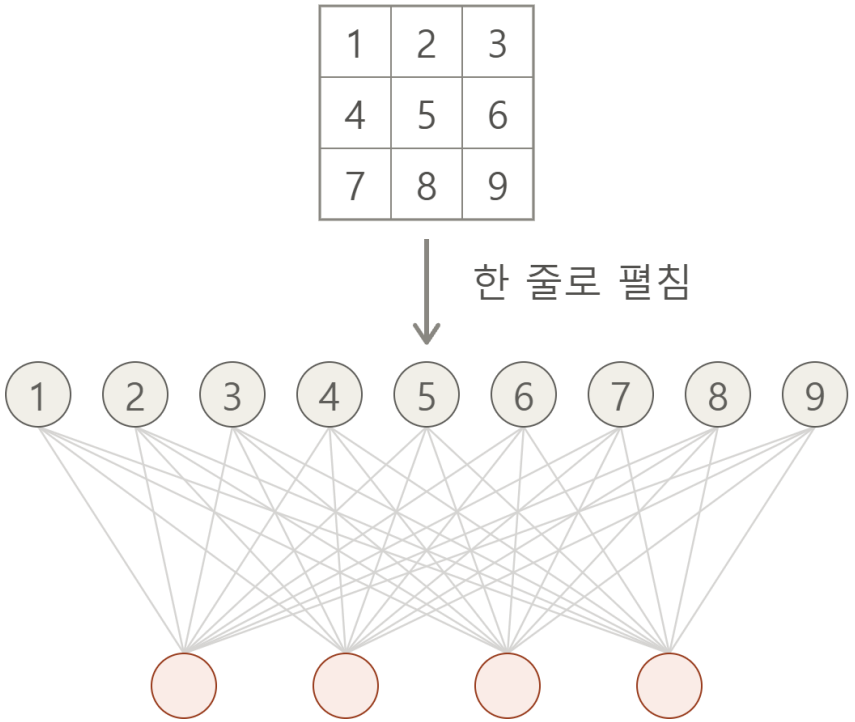
- **Darknet-19 backbone**
- **FC 제거, fully convolutional 구조**
- **Anchor box 도입**
- **Multi-scale training**
- **Passthrough layer로 fine-grained feature 활용**



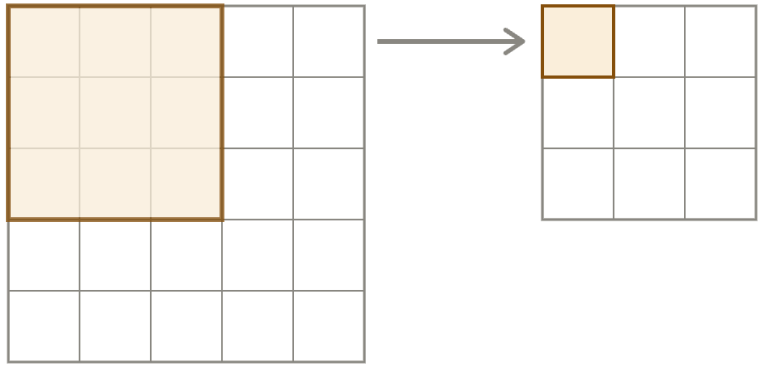
UNICON
Unmanned & Intelligent systems CONtrol LAB

## FC 층

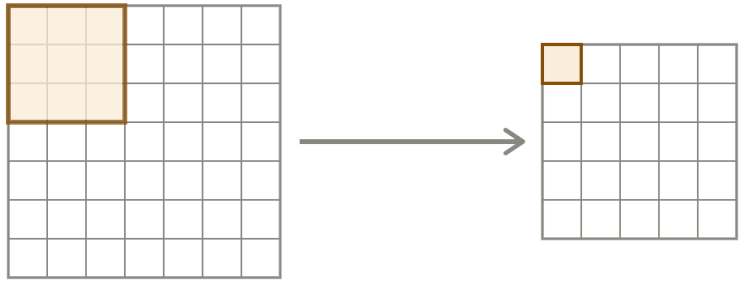



모든 값이 모든 출력과 연결
가중치 많음 · 위치 섞임 · 크기 고정

## Conv 층



입력이 더 커져도



같은 필터를 옮겨 가며 재사용
가중치 적음 · 위치 유지 · 크기 자유

UNICON
Unmanned & Intelligent systems CONtrol LAB

## YOLO 발전 흐름 (v1 → v12)

**v1** 2016
**S×S grid로 나눠 한 번에 예측**
→ single-stage 탐지의 시작

**v2** 2017
**Anchor box · Batch Norm · Darknet-19**
→ 정확도와 입력 크기 유연성 개선

**v3** 2018
**Darknet-53 + 3개 스케일 예측 (FPN)**
→ 작은 객체 탐지 향상

**v4** 2020
**CSPDarknet · PANet · Mosaic 증강**
→ 좋은 학습 기법의 집대성

**v5** 2020
**Ultralytics의 PyTorch 구현**
→ 쉬운 학습 · 배포 → 실무 표준

**v8** 2023
**Anchor-free · Decoupled head**
→ 탐지 · 분할 · 포즈를 하나의 프레임워크로

**v10** 2024
**NMS-free 학습 (one-to-one 할당)**
→ 후처리 지연 감소

**v11** 2024
**C3k2 · C2PSA 블록**
→ 더 적은 파라미터로 더 높은 mAP

**v12** 2025
**Area Attention · R-ELAN**
→ Attention 중심 실시간 탐지 (오늘 실습 모델)

UNICON
Unmanned & Intelligent systems CONtrol LAB

- **YOLOv12 – 구조적 변화**

**1)핵심 구조**

- **Attention-Centric YOLO**
- **Area Attention (A2)**
- **R-ELAN**
- **FlashAttention 활용**
- **Residual shortcut + scaling**

**2) 구조적 특징**

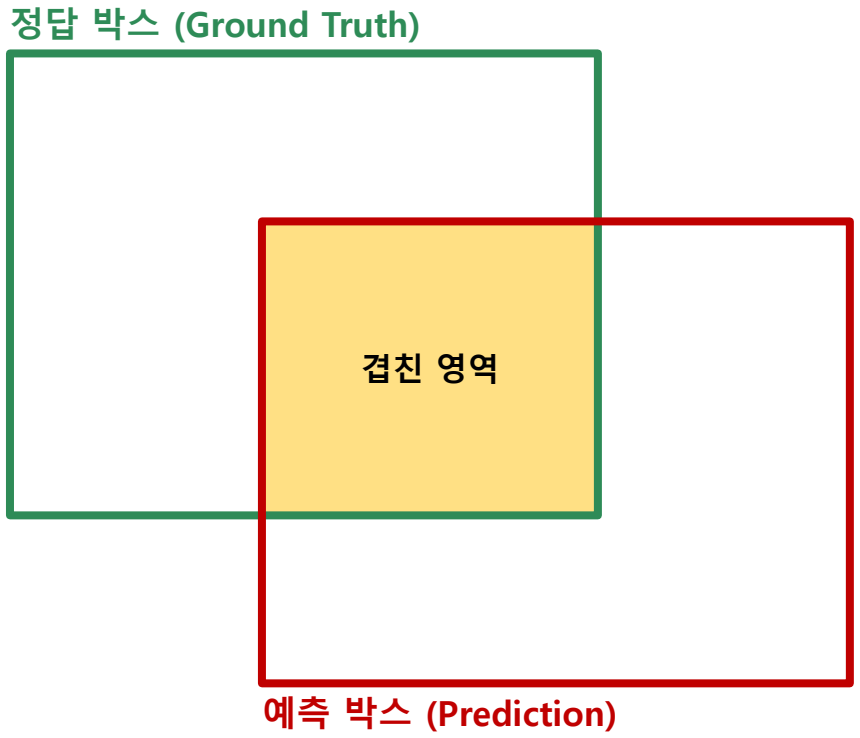
- CNN만이 아니라 attention을 본격 도입
- 더 넓은 receptive field 확보
- attention의 느린 속도를 구조적으로 최적화




Criss-cross attention | Window attention | Axial attention | Area attention (Ours)

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 핵심 개념 ① IoU (Intersection over Union)




$$IoU = \frac{\text{겹친 영역 (교집합)}}{\text{두 박스를 합친 영역 (합집합)}}$$

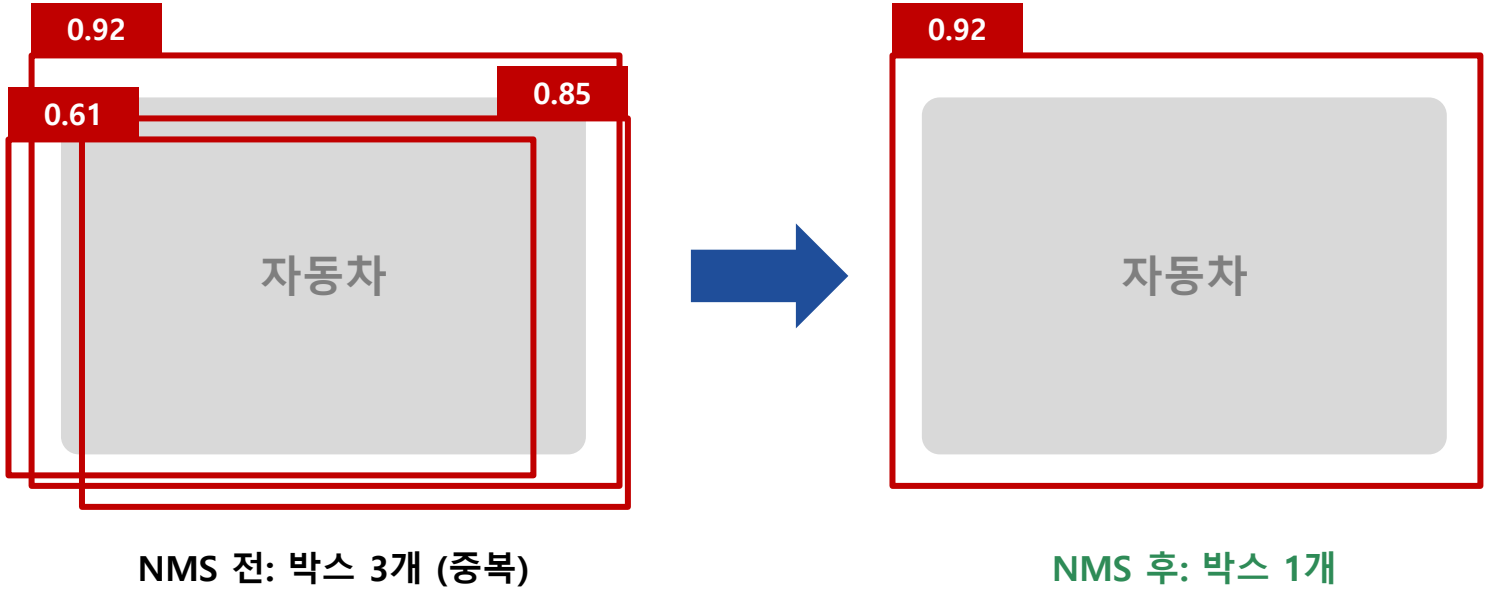
**IoU 0.9** → 거의 정확하게 일치
**IoU 0.5** → 정답으로 인정하는 기준선 (mAP50)
**IoU 0.22** → 왼쪽 예시: 위치가 많이 틀림

➔ 0 ~ 1 사이 값: 1이면 완전히 일치, 0이면 전혀 겹치지 않음
➔ mAP50-95는 IoU 기준을 0.50 ~ 0.95로 바꿔 가며 평균 → 더 엄격
➔ NMS에서도 IoU로 "같은 물체를 가리키는 박스인지" 판단

**이 예시: IoU = 2.24 ÷ 10.08 ≈ 0.22 → 오답(FP)**

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 핵심 개념 ② NMS (Non-Maximum Suppression)




**NMS 전: 박스 3개 (중복)**

**NMS 후: 박스 1개**

**실습 코드에서 조절**

```
model.predict(conf=0.25, iou=0.7)
```

➔ **conf**: 이 값보다 점수가 낮은 박스는 버림

➔ **iou**: 이 값보다 많이 겹치면 중복으로 보고 제거 (낮출수록 강하게 제거)

① conf가 기준값(0.25)보다 낮은 박스 제거

② 남은 박스 중 confidence가 가장 높은 박스 선택

③ 선택한 박스와 IoU가 0.7 이상인 같은 클래스 박스 삭제

④ 남은 박스로 ② ~ ③ 반복

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 핵심 개념 ③ Precision · Recall

**TP** — **True Positive**
물체가 있는 곳을 맞게 찾음 (IoU ≥ 기준)

**FP** — **False Positive**
없는 곳에 박스를 침 → 오검출

**FN** — **False Negative**
있는 물체를 놓침 → 미검출

※ 객체 탐지에서는 '배경을 배경이라고 맞힌 것(TN)'은 셀 수 없어서 쓰지 않습니다

**Precision (정밀도) = TP ÷ (TP + FP)**
모델이 찾은 것 중에 맞은 비율 → "헛것을 덜 보는가?"

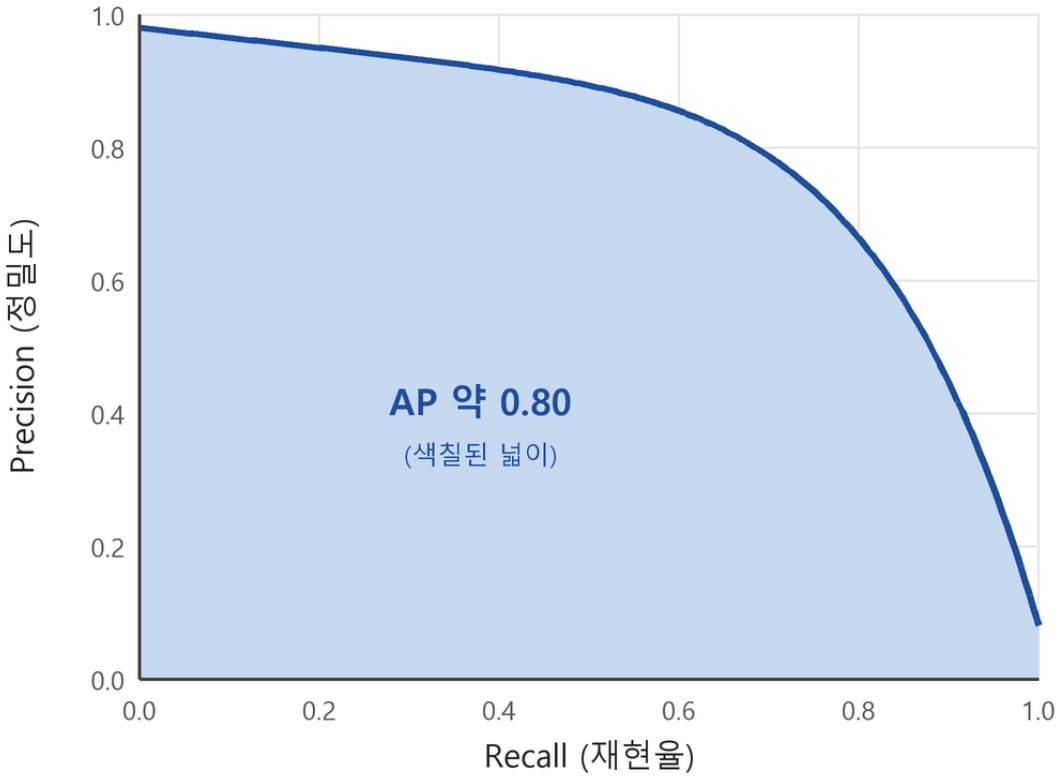
**Recall (재현율) = TP ÷ (TP + FN)**
실제 물체 중에 찾아낸 비율 → "놓치지 않는가?"

**예시) 실제 자동차 10대 · 모델이 친 박스 8개 · 그중 6개 정답**
Precision = 6 ÷ 8 = 0.75  Recall = 6 ÷ 10 = 0.60

**conf를 낮추면 Recall ↑ Precision ↓, 높이면 반대 → 균형이 중요**

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 핵심 개념 ④ mAP (mean Average Precision)




**P-R 곡선 아래 넓이 = AP (클래스 하나의 점수)**

① conf 기준을 높게 → 낮게 바꾸며 Precision · Recall을 계산해 P-R 곡선을 그림

② 곡선 아래 넓이 = AP (클래스 하나의 점수)

③ 모든 클래스의 AP 평균 = mAP

**mAP50** IoU 0.5 이상이면 정답 인정 (느슨함)

**mAP50-95** IoU 0.50 ~ 0.95, 10개 기준의 평균 (엄격 · 대표 지표)

**46번 검증 결과 읽기**

Precision 0.686 · Recall 0.714 · mAP50 0.751 · mAP50-95 0.560

# 2. YOLO v12 algorithm 실습

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [1 / 8]**




- **Computer Vision Application**을 구축하고 배포하는데 필요한 툴과 서비스를 제공하는 플랫폼
- **Machine Learning**과 **AI(Artificial Intelligence)**를 활용한 이미지 및 영상 데이터 처리에 용이

➔ **데이터셋 관리** : 다양한 이미지 데이터를 손쉽게 업로드, 레이블링, 관리할 수 있는 인터페이스 제공

➔ **데이터 라벨링** : 사용자가 이미지를 라벨링하고 주석을 달 수 있도록 직관적인 툴 제공(객체 감지, 분류 등)

➔ **데이터 증강** : 기존 데이터셋을 다양하게 변형하여 확장할 수 있도록 하는 데이터 증강 툴 제공

➔ **모델 훈련** : 자신의 데이터셋을 기반으로 Machine Learning 모델을 훈련시킬 수 있도록 지원

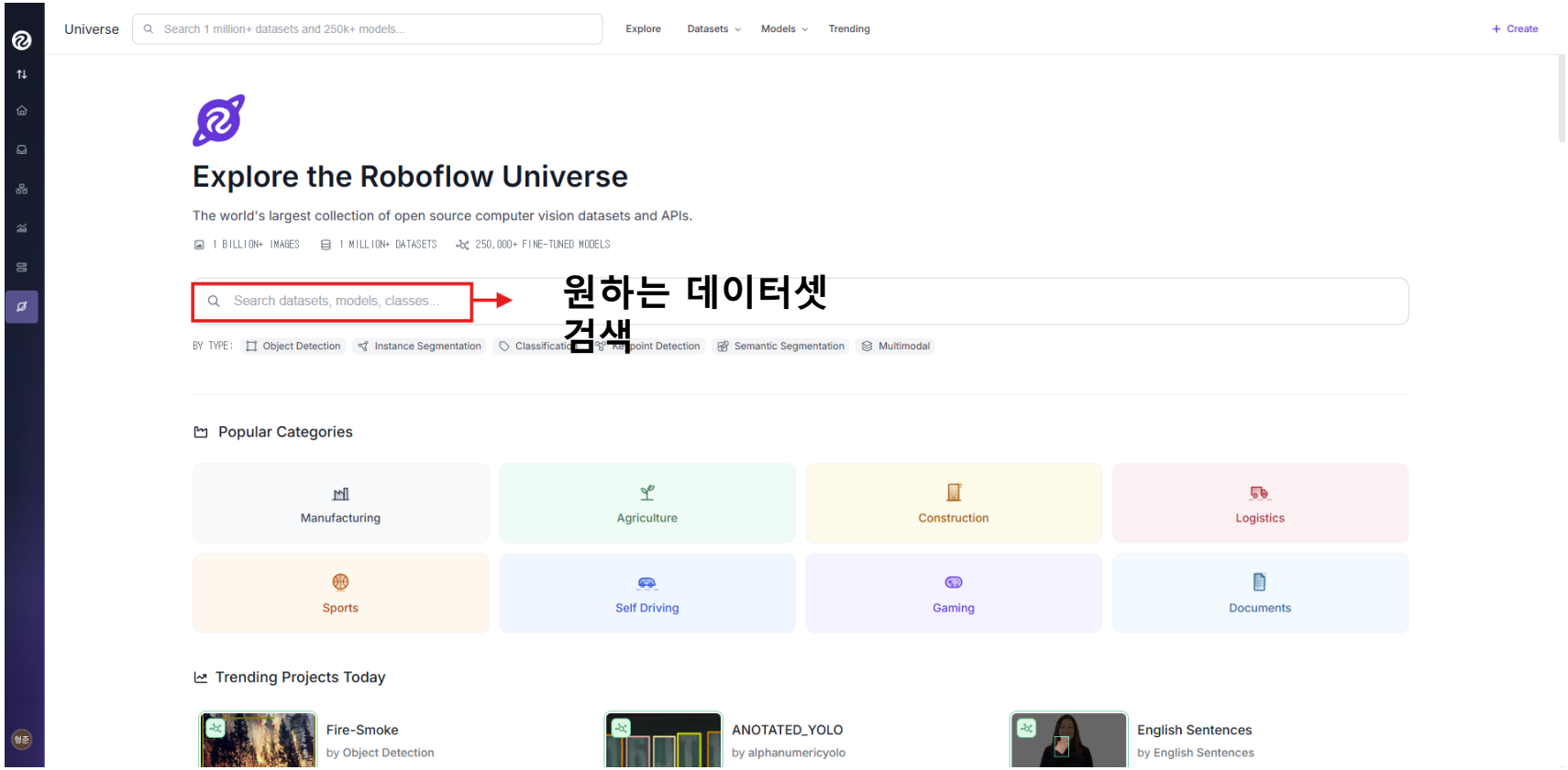
➔ **모델 배포** : 자신의 데이터셋을 기반으로 훈련된 모델을 쉽게 배포하고 관리할 수 있는 툴 제공

UNICON
Unmanned & Intelligent systems CONtrol LAB

# 1) YOLO v12 학습을 위한 Dataset 구축

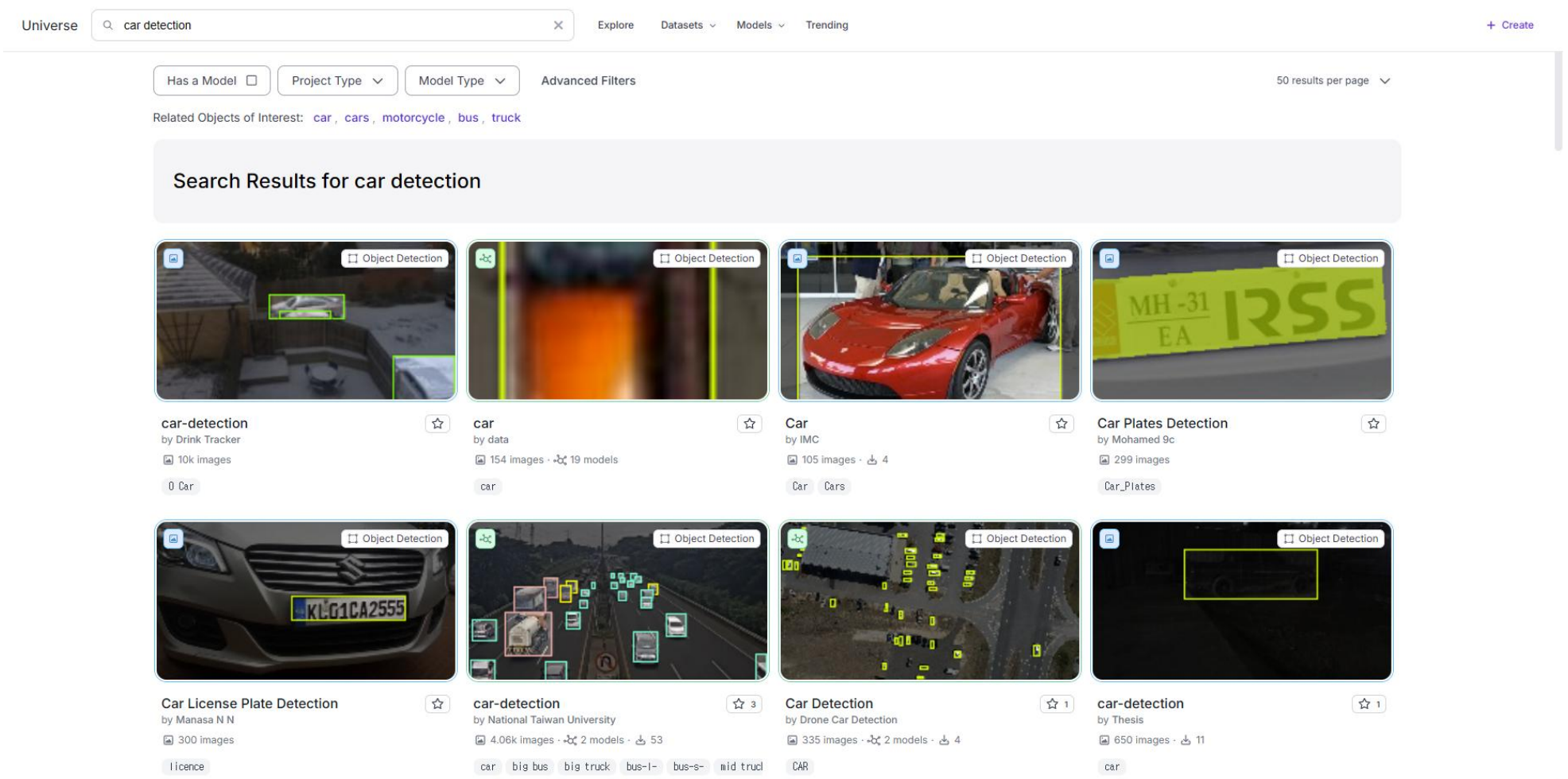
- **Roboflow Dataset [2 / 8]**

➔ https://universe.roboflow.com/ **(해당 링크에 접속하여 원하는 데이터셋 검색)**



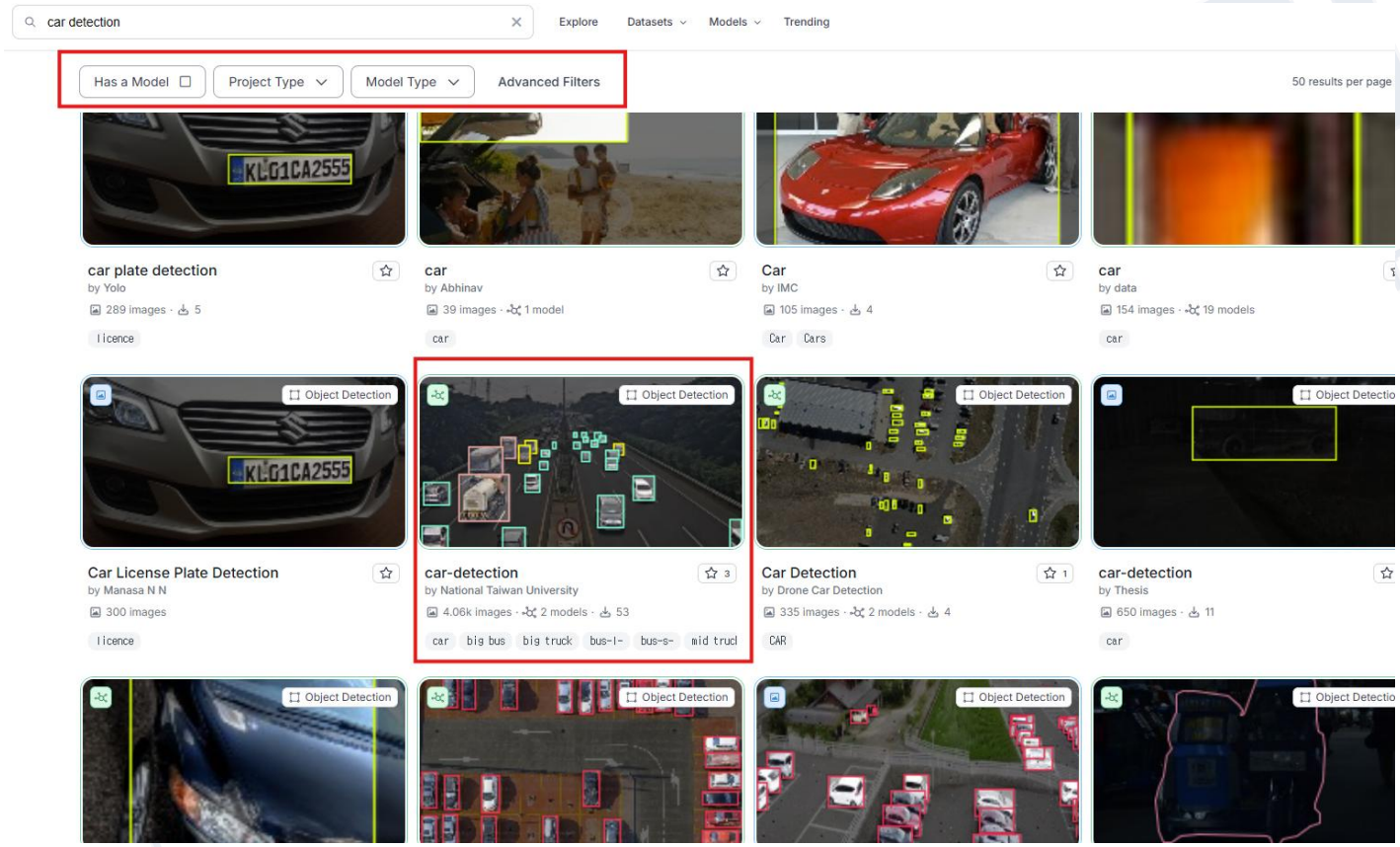
# 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [3 / 8]**



# 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [4 / 8]**

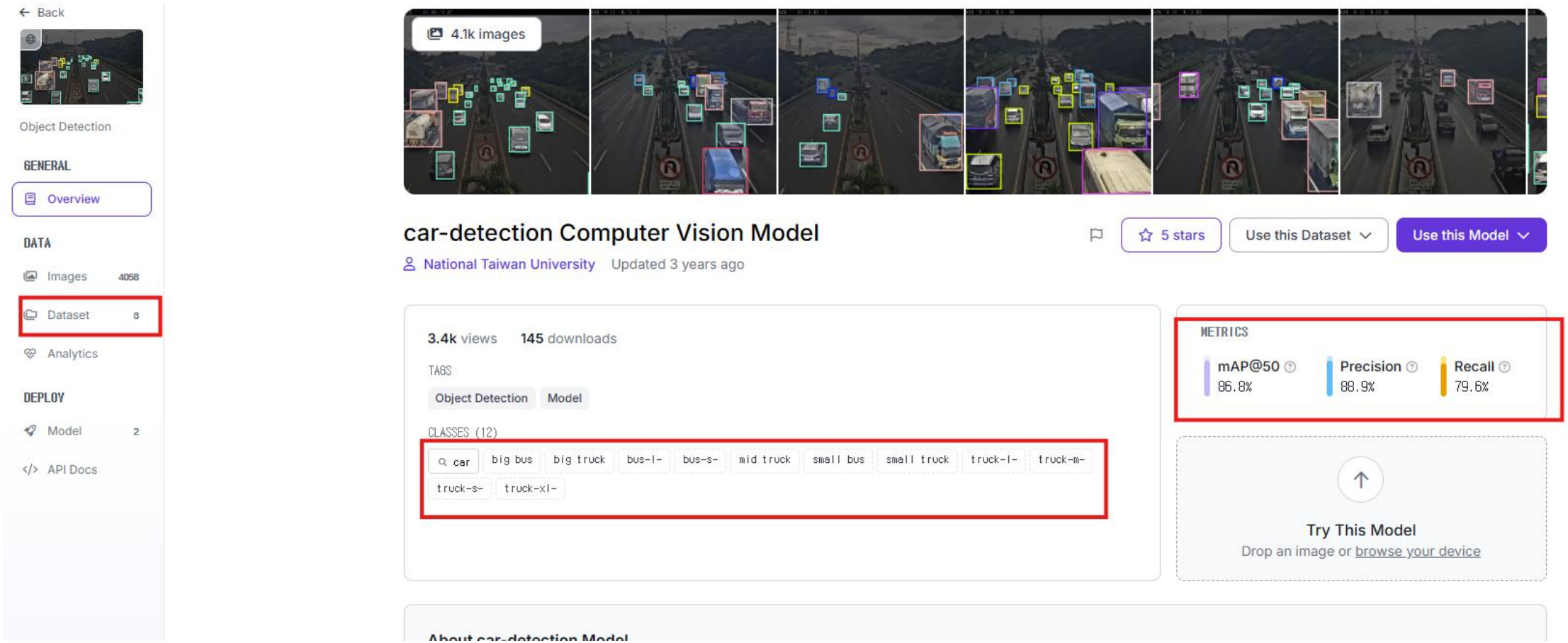



https://universe.roboflow.com/national-taiwan-university-m64jc/car-detection-9mxpz

UNICON
Unmanned & Intelligent systems CONtrol LAB

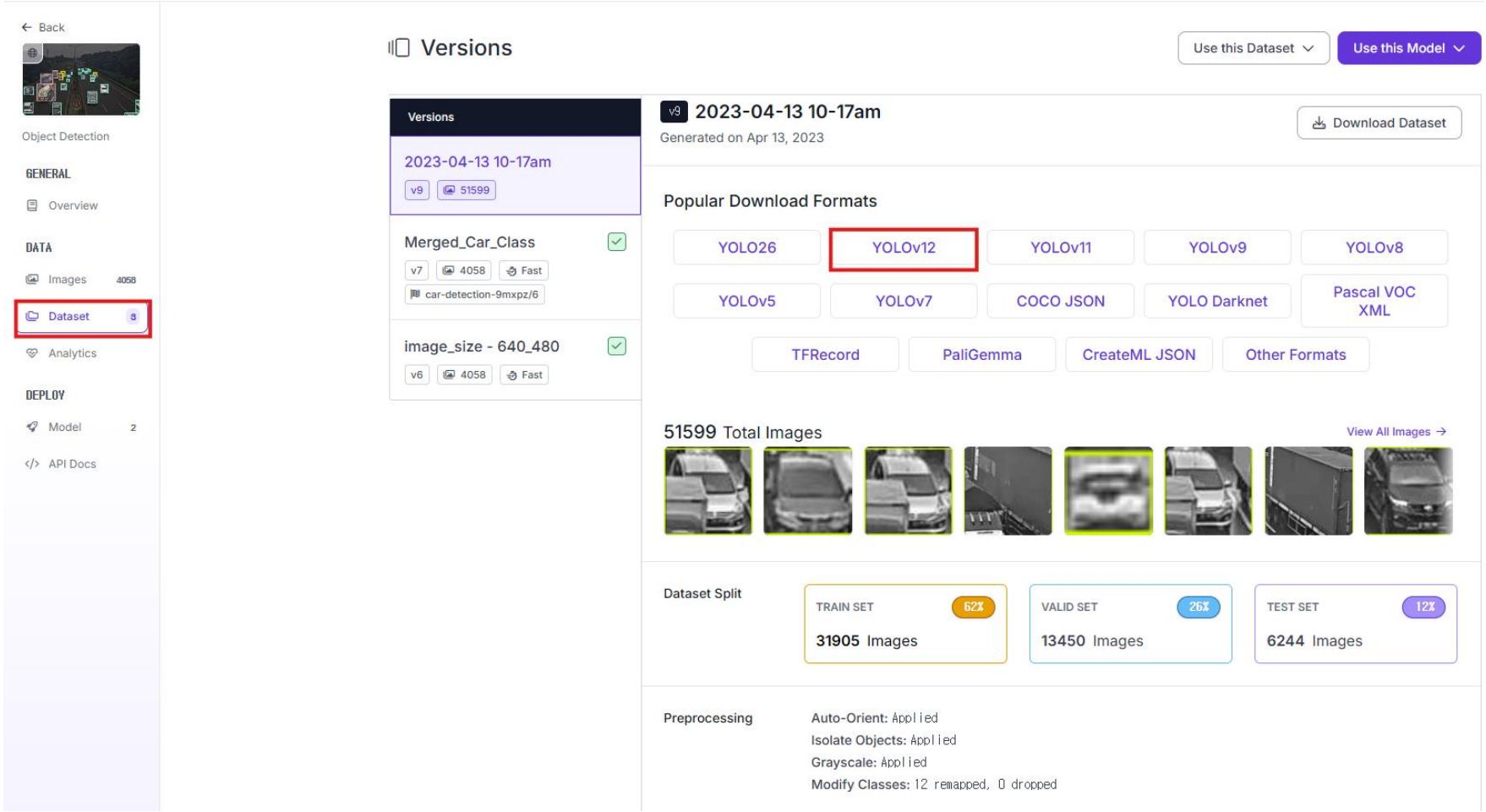
# 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [5 / 8]**



# 1) YOLO v12 학습을 위한 Dataset 구축

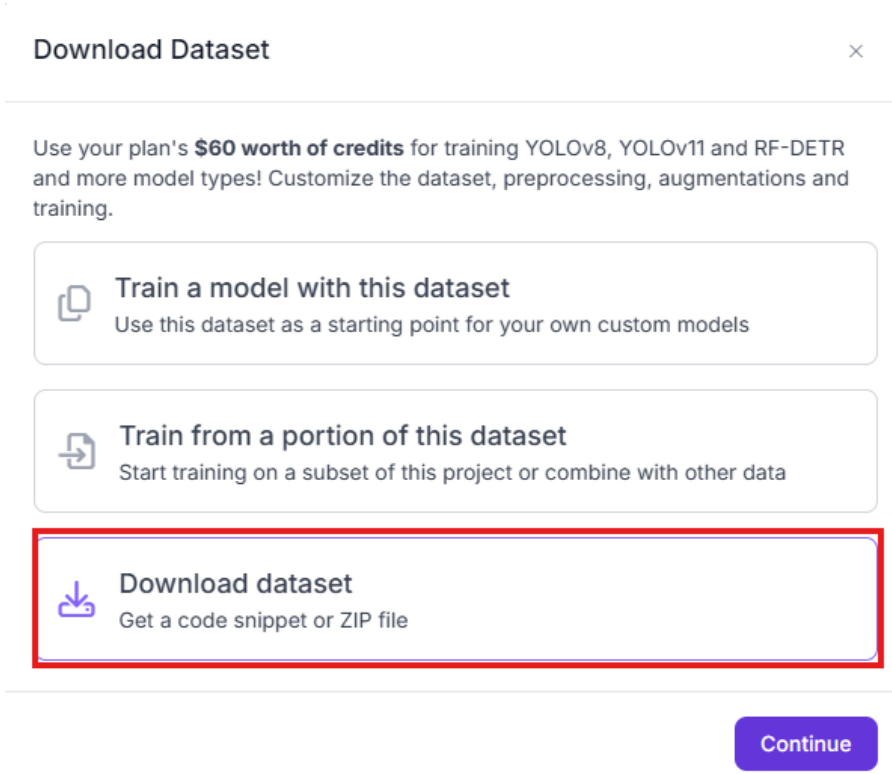
- **Roboflow Dataset [6 / 8]**

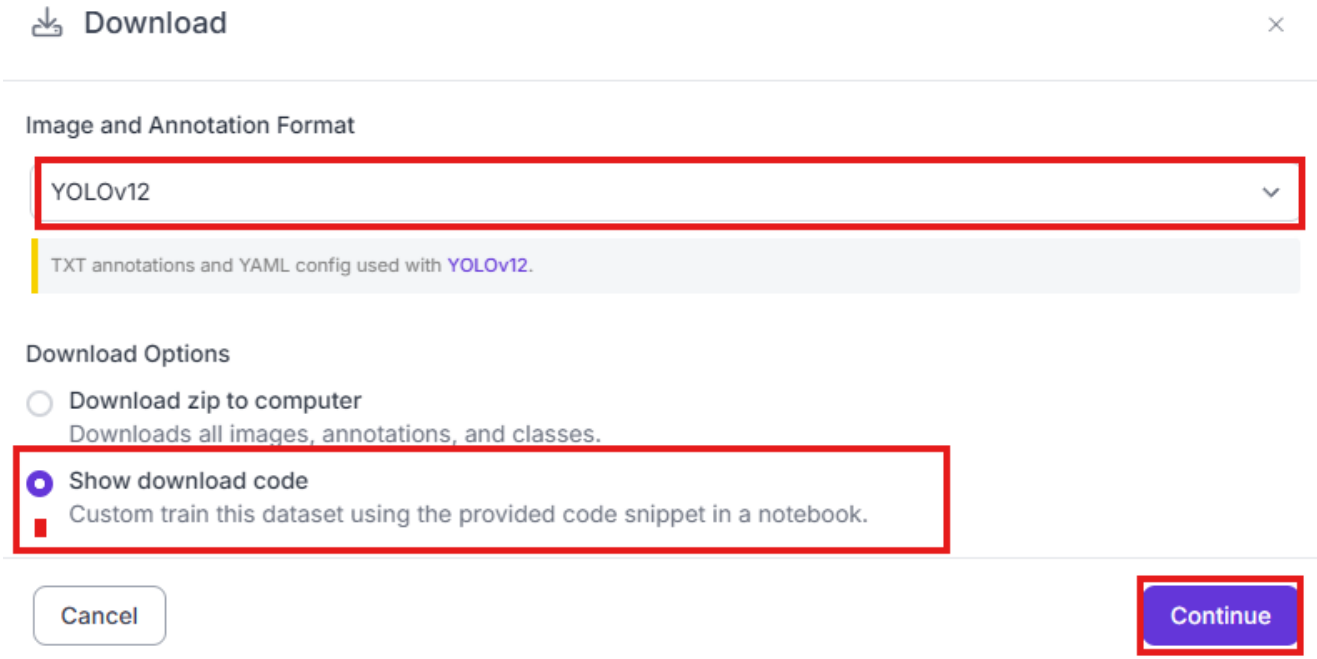


UNICON
Unmanned & Intelligent systems CONtrol LAB

# 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [7 / 8]**

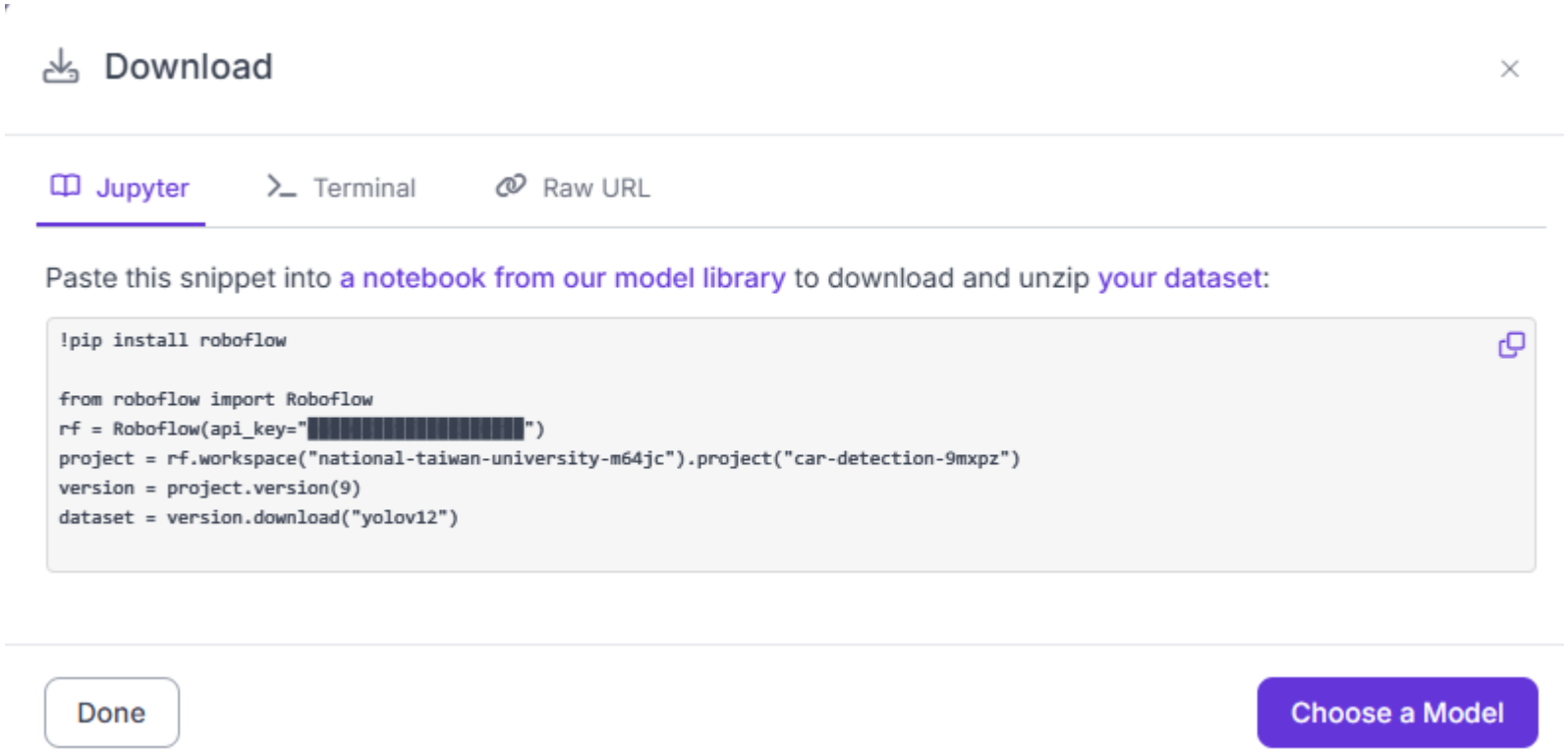


UNICON
Unmanned & Intelligent systems CONtrol LAB

## 1) YOLO v12 학습을 위한 Dataset 구축

- **Roboflow Dataset [8 / 8]**




➔ 해당 페이지의 Download Code까지 활성화 시켰다면, Dataset 준비가 완료된 것.

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 구축 [1 / 4]**



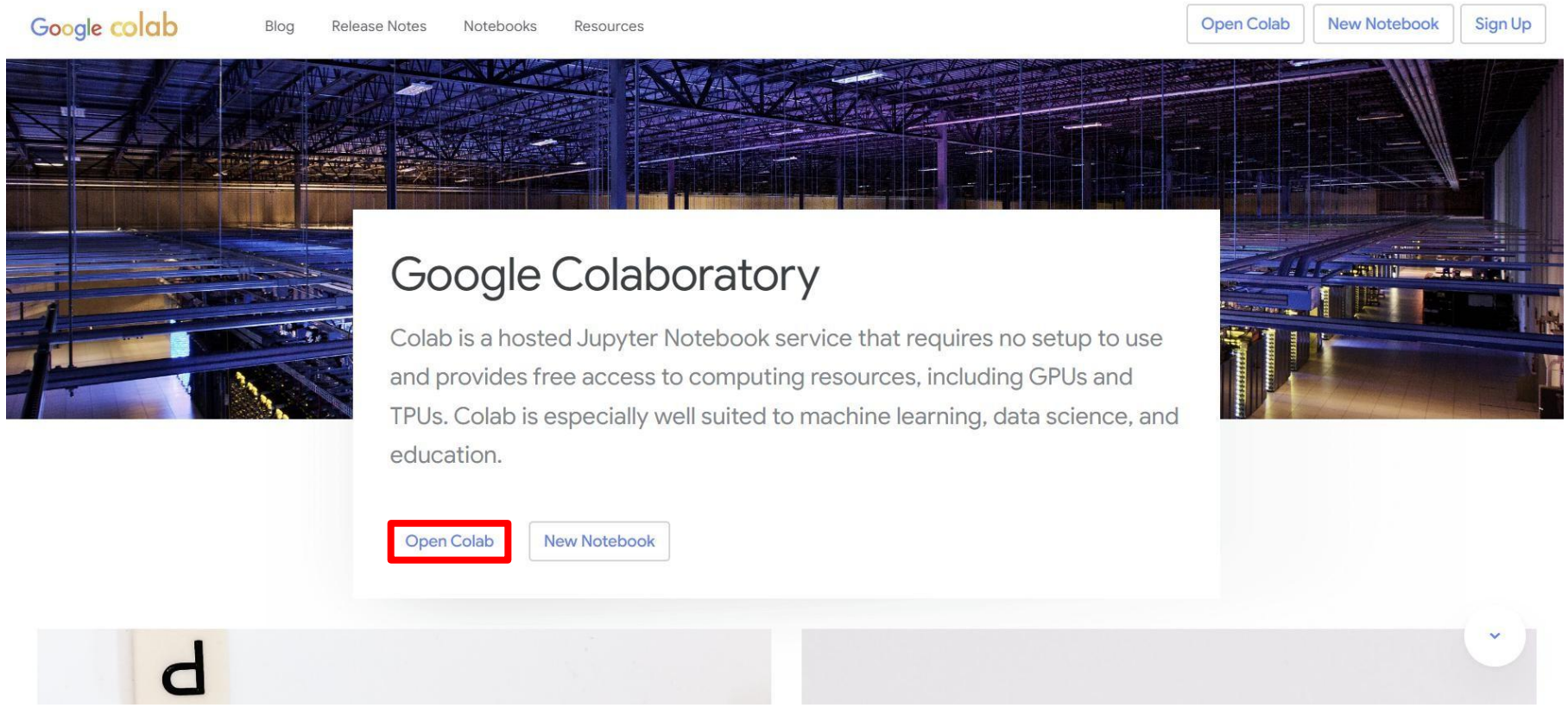

- **Google Colab**은 **Google**에서 제공하는 **클라우드 기반의 Jupyter 노트북 환경**
- **Colab**을 통해 사용자는 **별도의 소프트웨어 설치 없이** 웹 브라우저에서 코드를 작성하고 실행 가능
- 가장 큰 특징으로 **GPU**, **TPU**를 무료로 사용 가능 (단, **리소스가 제한**되어 있음)
- Data Science, Machine Learning, Deep Learning과 같은 프로젝트에 유용

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 구축 [2 / 4]**

  ➔ https://colab.google/ **(해당 링크에 접속하여 Google Colab 실행)**

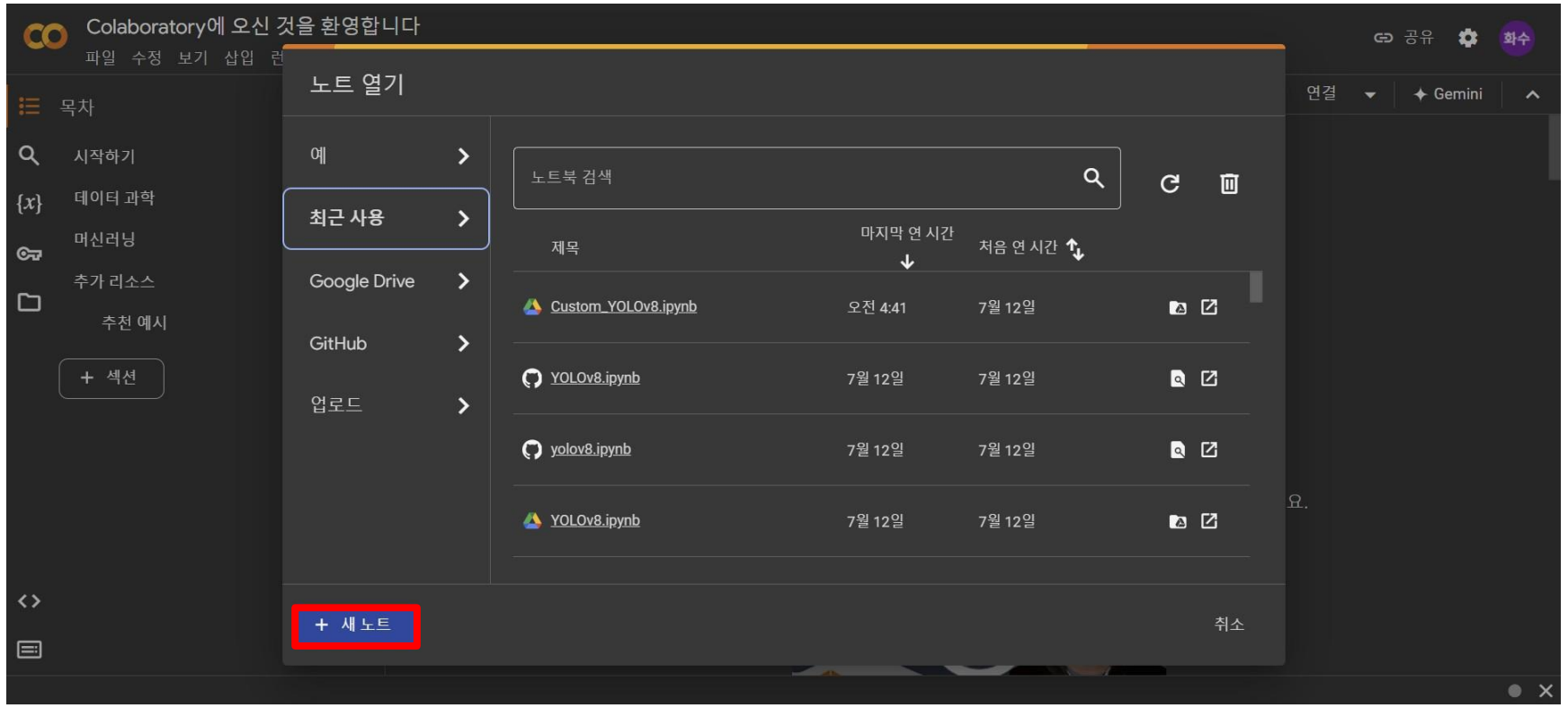


UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 구축 [3 / 4]**

  **➔ Google Colab 실행 후 , Google 로그인 및 “새 노트” 클릭하여 Colab 환경 구축**

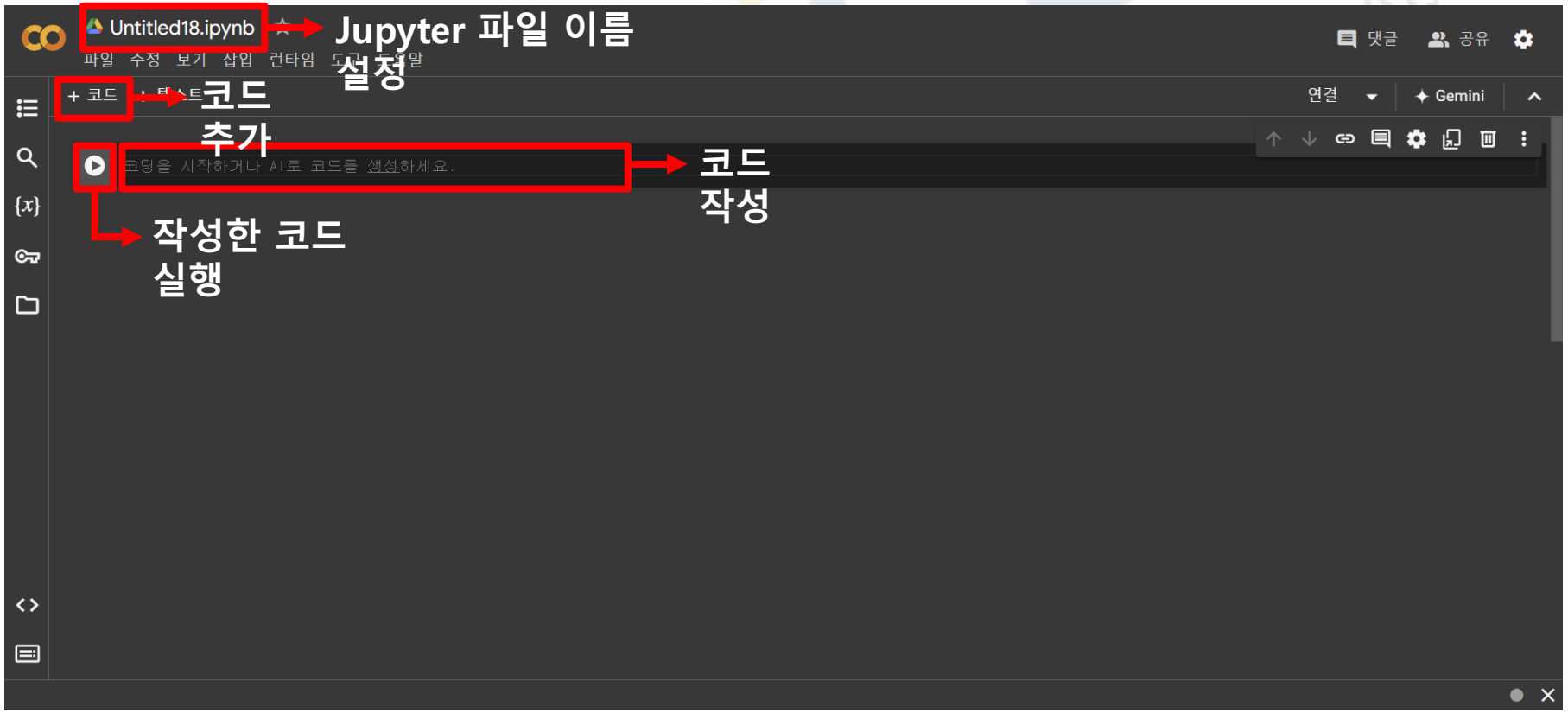


UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

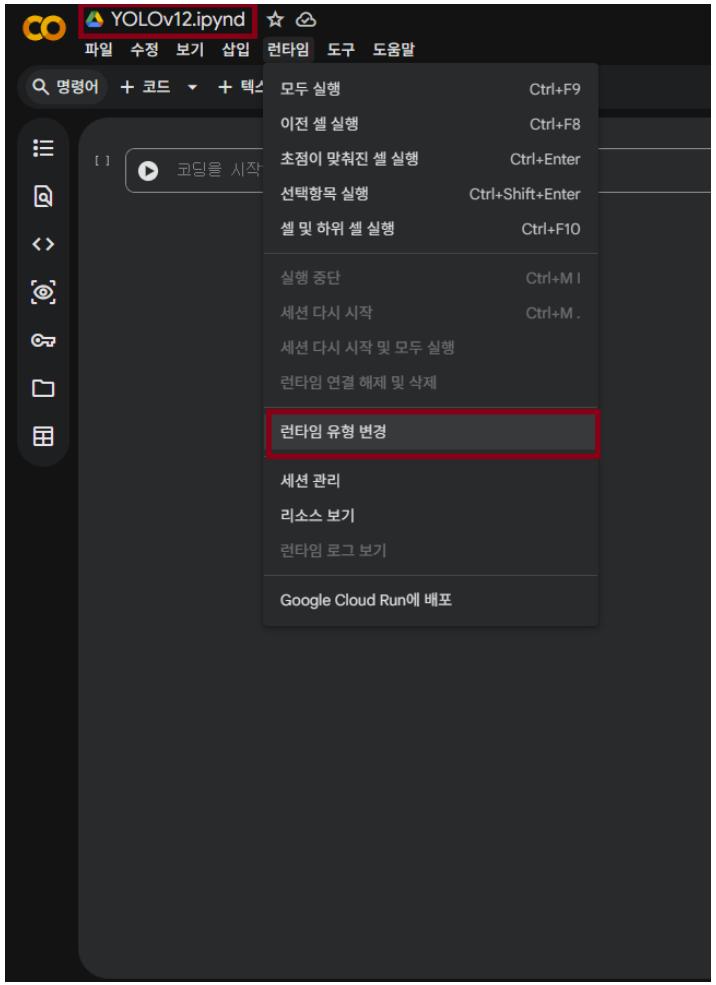
- **Google Colab 환경 구축 [4 / 4]**

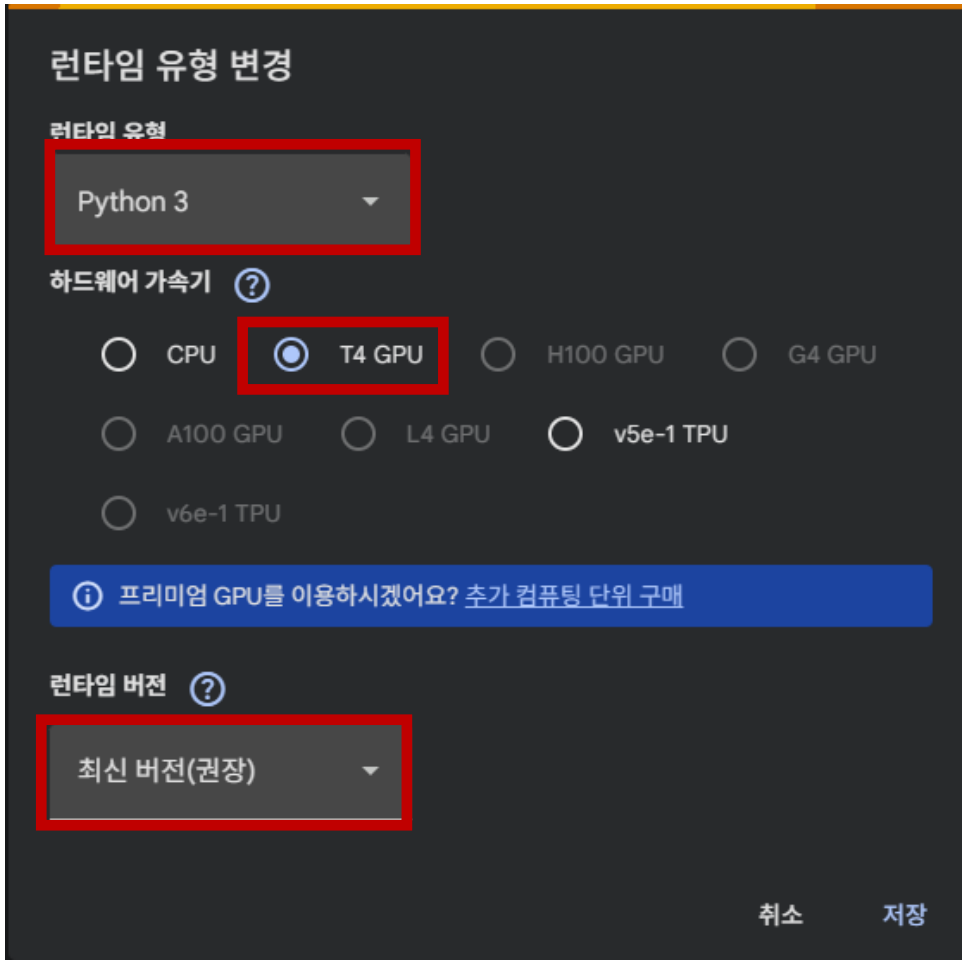
  **➔ Google Colab 실행 후 , Google 로그인 및 “새 노트” 클릭하여 Colab 환경 구축**



## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 GPU/TPU 설정 [1 / 2]**

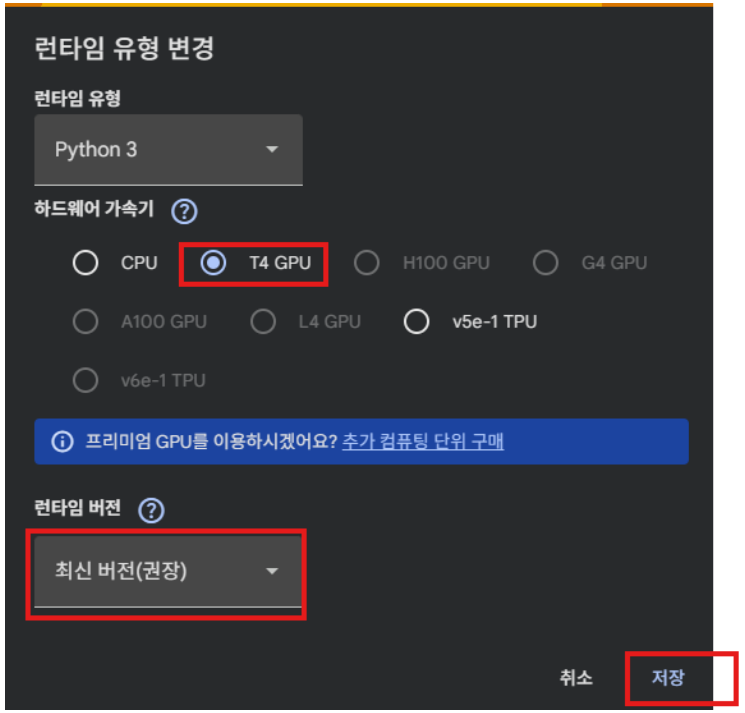


UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 GPU/TPU 설정 [2 / 2]**






UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 설치 [1 / 2]**

```
!pip install ultralytics #YOLO 모델 라이브러리
```

숨겨진 출력 표시

```
from ultralytics import YOLO

model = YOLO('yolo12n.pt') #YOLO12의 nano 버전
```

```
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo12n.pt to 'yolo12n.pt': 100% ━━━━━━━━━━━━ 5.3MB 109.4MB/s 0.0s
```

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 설치 [2 / 2]**

```
import torch

model = torch.load("yolo12n.pt", map_location='cpu', weights_only=False)
print(model['model'].names)
```

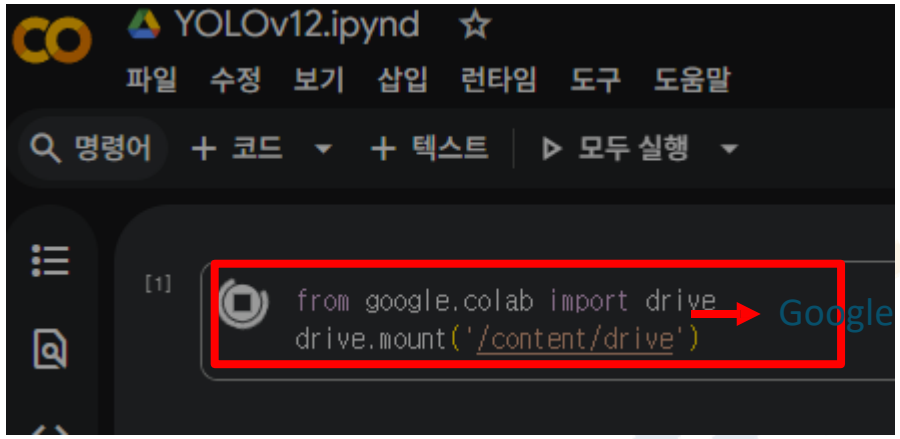


{0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 16: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'tie', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports ball', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfboard', 38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife', 44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich', 49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza', 54: 'donut', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dining table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard', 67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'toaster', 71: 'sink', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors', 77: 'teddy bear', 78: 'hair drier', 79: 'toothbrush'}

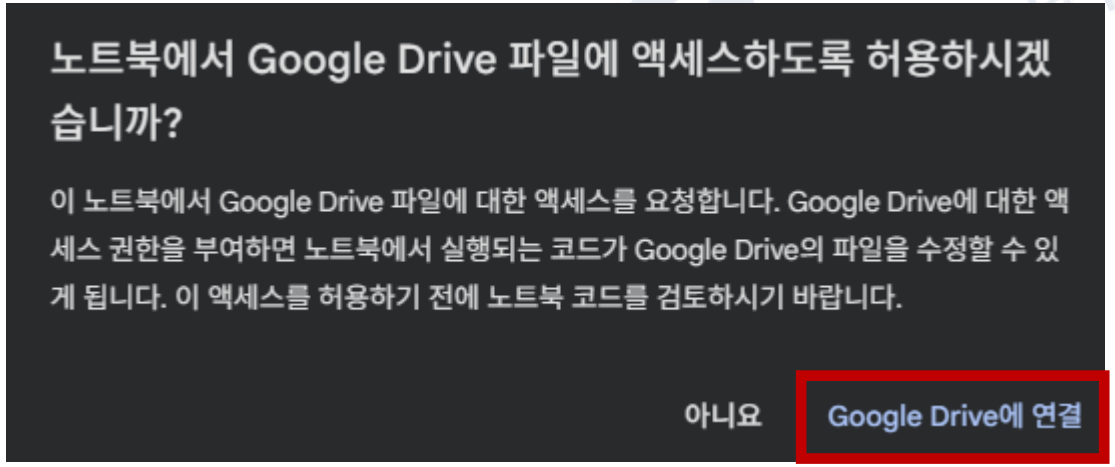
UNICON
Unmanned & Intelligent systems CONtrol LAB

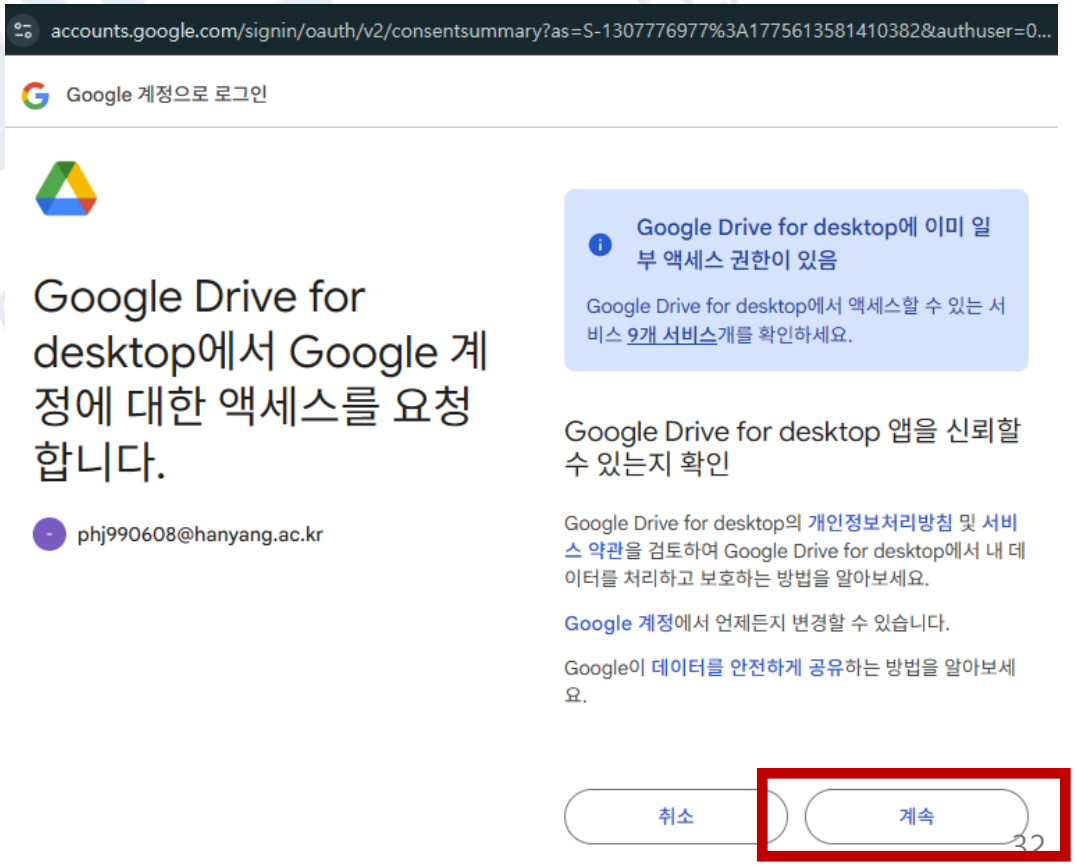
## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경과 Google Drive 연동 [1 / 2]**




Google Drive와 Google Colab 연동

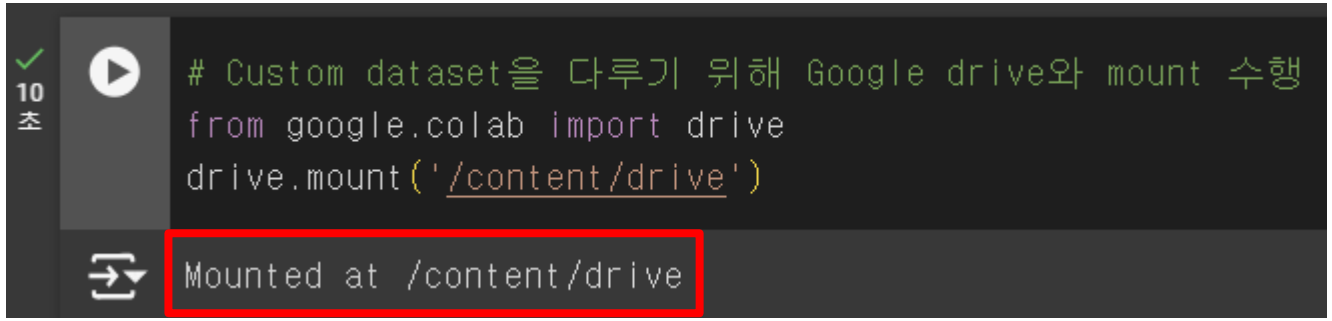


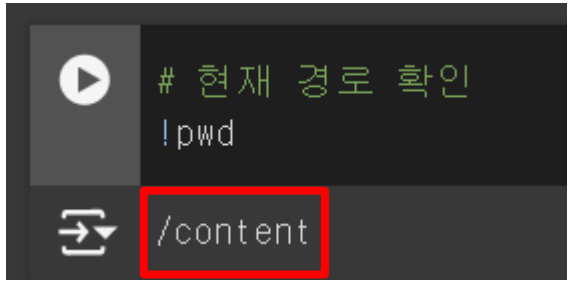



UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경과 Google Drive 연동 [2 / 2]**

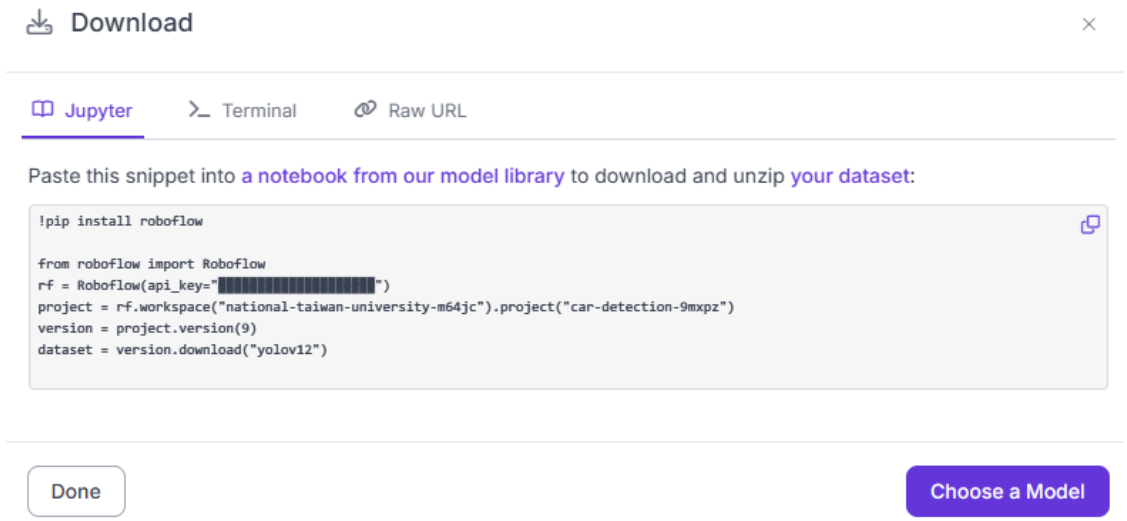


UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 다운로드 [1 / 3]**




```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("suvsed").project("car-models-5iepb")
version = project.version(26)
dataset = version.download("yolov12")
```

→ 터미널에서 pwd 로 현재 파일 위치 확인
→ cd /content 안에서 실행

**※ api_key에는 각자 본인 Roboflow 계정의 키를 넣으세요.** Download Code(23번)를 복사하면 본인 키가 자동으로 들어갑니다. 키는 외부에 공유하지 마세요.

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 다운로드 [2 / 3]**




**폴더명 변경**

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 다운로드 [3 / 3]**

```
0초
# /content/roboflow 디렉토리의 파일 확인
!ls /content/roboflow

data.yaml  README.dataset.txt  README.roboflow.txt  test  train  valid
```

```
train: C:/Users/hsym3/ultralytics/roboflow/train/images
val: C:/Users/hsym3/ultralytics/roboflow/valid/images
test: C:/Users/hsym3/ultralytics/roboflow/test/images

nc: 1  # number of classes
names: ['car']  # class names
```

**data.yaml 파일 예시**

```
roboflow/
    train/
        images/
        labels/
    valid/
        images/
        labels/
    test/
        images/
        labels/
```

**※ 위 data.yaml은 로컬(Windows) 예시입니다**

Colab에서는 경로를 아래처럼 바꿔야 합니다:

```
train: /content/roboflow/train/images
val:   /content/roboflow/valid/images
test:  /content/roboflow/test/images
```

nc · names는 내 데이터셋의 클래스와 똑같이 맞추세요

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [1 / 7]**

```python
from ultralytics import YOLO
import os
#필요한 디렉토리 생성
os.makedirs('/content/runs', exist_ok=True)
model = YOLO('yolo12n.pt') # YOLO모델사용
model.train(
    data='/content/roboflow/data.yaml', #데이터셋 설정 경로 파일
    epochs=3, #학습 epoch 수
    imgsz=256, #입력 이미지 크기
    batch=16, #배치 크기
    workers = 4, #데이터 로딩을 위한 병렬 작업 수
    fraction=0.3, #일부 데이터만 학습. 로딩할때 랜덤으로 30%만 사용하여 학습 0~1
    project = '/content/runs', #결과를 저장할 디렉토리
    name='exp' #시험 이름
)
```

**※ 시연용 설정입니다**

➔ epochs=3 · imgsz=256 · fraction=0.3 → 동작 확인용으로 짧게 설정

➔ 실습 과제에서는 조건에 맞게 epochs를 늘리고 imgsz=640을 권장

➔ GPU 사용 시간이 부족하면 fraction · imgsz를 줄여서 조절

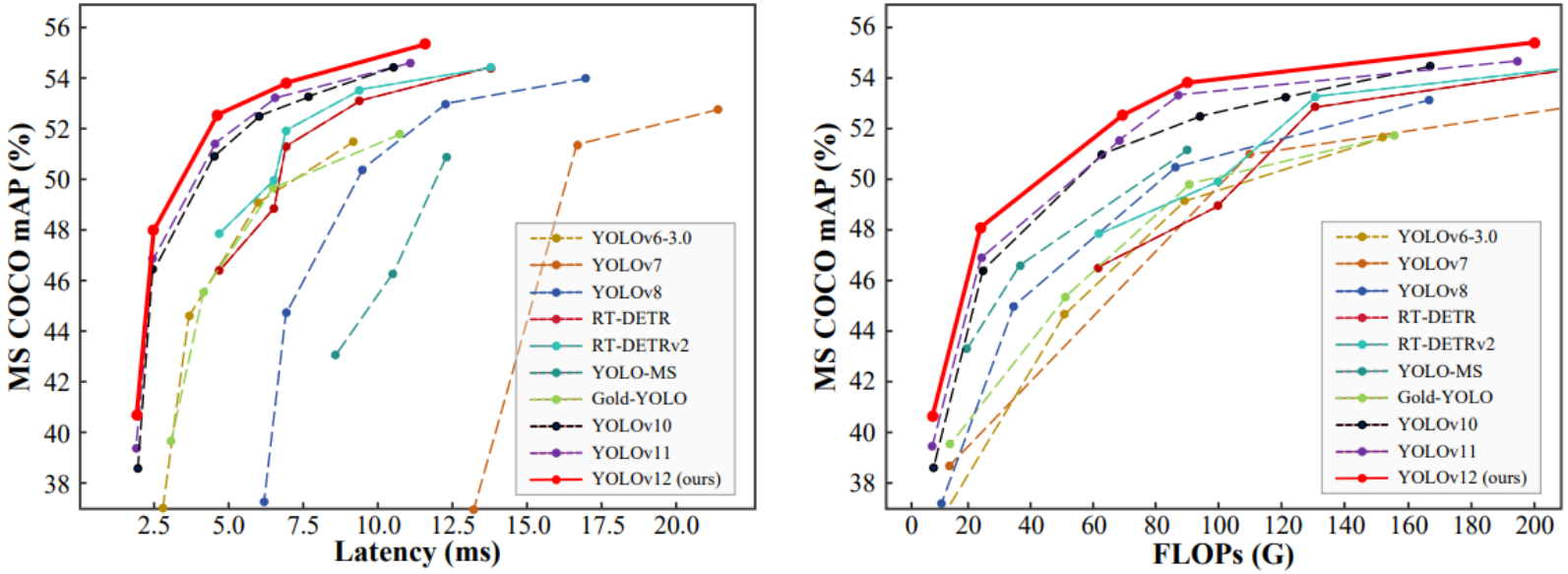
➔ 같은 name으로 다시 실행하면 결과 폴더가 exp2, exp3 …으로 늘어남 (덮어쓰려면 exist_ok=True)

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  1/3      0.584G      1.999      4.487        1.2        127        256: 100% ──────────── 50/50 1.9it/s 25.9s
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 31/31 2.9it/s 10.7s
               all        966      13450    0.00678      0.145    0.00784    0.00292

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  2/3      0.586G      1.833      3.798      1.029        214        256: 100% ──────────── 50/50 6.3it/s 7.9s
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95):  26% ──────────── 8/31 6.1it/s 1.2s<3.8s
```

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [2 / 7]**




➔ **YOLO v12 모델은** Area Attention Mechanism을 도입하여 CNN에 필적하는 속도와 CNN을 뛰어넘는 정확성 확보

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [3 / 7]**

| 모델명 | 학습 속도 | mAP (성능) | 활용 특징 |
|---|---|---|---|
| N | 매우 빠름 | 낮음 | 실험, edge |
| S | 빠름 | 중간 | 실용 |
| M | 보통 | 높음 | 연구 |
| L | 느림 | 더 높음 | 고성능 |
| X | 매우 느림 | 최고 | benchmark용 |

# 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [4 / 7]**

```
Logging results to /content/runs/exp
Starting training for 50 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/50      2.45G      3.136      3.556      4.172         24        640: 100%|██████████| 55/55 [00:23<00:00,  2.31it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:03<00:00,  1.73it/s]                   all        187


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/50      2.25G      3.045      3.247      3.716         24        640: 100%|██████████| 55/55 [00:19<00:00,  2.78it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:03<00:00,  1.85it/s]                   all        187


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       3/50      2.26G      2.814      3.035      3.378         24        640: 100%|██████████| 55/55 [00:19<00:00,  2.89it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:04<00:00,  1.45it/s]                   all        187


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       4/50      2.26G      2.549       2.79      3.178         30        640: 100%|██████████| 55/55 [00:18<00:00,  3.00it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:03<00:00,  1.62it/s]                   all        187
```

➔ 설정한 GPU를 기반으로 총 N epoch를 반복하여 학습 수행

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [5 / 7]**

  ➔ 만약 GPU를 사용하지 않고, CPU를 사용하게 되면 학습 소요 시간 대폭 상승

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 1/50         0G      3.131      3.583      4.174         31        640: 100%|██████████| 55/55 14:15<00:00 15.56s/it]
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [01:09<00:00, 11.57s/it]

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 2/50         0G      3.012      3.377      3.751         35        640: 100%|██████████| 55/55 12:46<00:00 13.94s/it]
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:57<00:00,  9.63s/it]
```

CPU 기반 YOLO 학습

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 1/50       2.59G      3.136      3.556      4.172         24        640: 100%|██████████| 55/55 00:23<00:00  2.33it/s]
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:04<00:00,  1.37it/s]

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 2/50       2.25G      3.045      3.247      3.716         24        640: 100%|██████████| 55/55 00:20<00:00  2.73it/s]
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:04<00:00,  1.29it/s]
```

GPU 기반 YOLO 학습

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [6 / 7]**

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      49/50      2.25G     0.6917     0.5002      1.265         11        640: 100%|██████████| 55/55 [00:18<00:00,  2.98it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:02<00:00,  2.79it/s]                   all        187


      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      50/50      2.25G     0.6795     0.4949      1.244         14        640: 100%|██████████| 55/55 [00:18<00:00,  3.01it/s]
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:01<00:00,  3.20it/s]                   all        187


50 epochs completed in 0.320 hours.
Optimizer stripped from /content/runs/exp/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/exp/weights/best.pt, 6.2MB

Validating /content/runs/exp/weights/best.pt...
Ultralytics YOLOv8.2.57 🚀 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
YOLOv8n summary (fused): 168 layers, 3,005,843 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 6/6 [00:06<00:00,  1.06s/it]
                   all        187        238      0.979      0.929      0.982      0.797
Speed: 1.0ms preprocess, 3.6ms inference, 0.0ms loss, 7.9ms postprocess per image
Results saved to /content/runs/exp
```

**※ 참고**
위 화면은 YOLOv8n으로 학습한 로그 예시입니다. YOLO12n도 출력 형식은 같고, 요약 줄에 YOLO12n으로 표시됩니다.

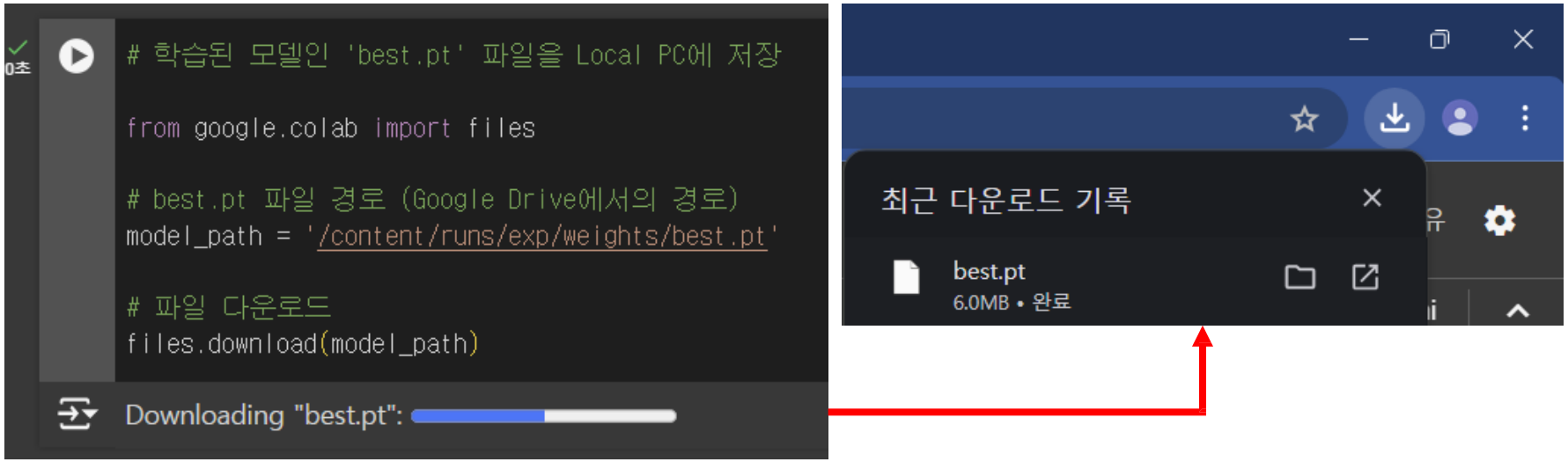
➔ 학습 과정 중 특정 지표(대표적으로 'mAP')가 향상될 때마다 모델의 상태를 저장하며, 가장 좋은 성능을 보였을 때의 가중치를 'best.pt'로 저장

➔ 학습된 모델의 저장 경로는 '/content/runs/exp/weights/best.pt'

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 2) YOLO v12 기반 객체 인지 모델 학습

- **Google Colab 환경 내 YOLO v12 Dataset 학습 [7 / 7]**




➔ 다음의 경로 '/content/runs/exp/weights/best.pt'에 저장되어 있는 학습된 모델을 Local PC에 저장

➔ 다운로드한 모델 'best.pt' 파일을 통해 어디서든 학습된 모델을 불러올 수 있음

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [1 / 6]**

```
from ultralytics import YOLO
import matplotlib.pyplot as plt
import seaborn as sns
import cv2

# 학습된 모델 로드
model = YOLO('/content/runs/exp/weights/best.pt')

# 검증 데이터셋 설정 (예: coco128.yaml 파일)
validation_data = '/content/roboflow/data.yaml'

# 모델 검증
metrics = model.val(data=validation_data)
```

➔ 다음의 경로 '/content/runs/exp/weights/best.pt'에 저장되어 있는 학습된 모델을 load하여 data.yaml 파일에 위치한 validation data의 경로를 통해 validation 수행

UNICON
Unmanned & Intelligent systems CONtrol LAB

# 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [2 / 6]**

```python
from ultralytics import YOLO
import matplotlib.pyplot as plt
import seaborn as sns
import cv2

# 학습된 모델 로드
model = YOLO('/content/runs/exp7/weights/best.pt')

# 검증 데이터셋 설정 (예: coco128.yaml 파일)
validation_data = '/content/roboflow/data.yaml'

# 모델 검증
metrics = model.val(data=validation_data)

# 성능 지표 출력
# 'Precision' = 정확도
#  => 'Precision'은 모델이 예측한 객체 중 실제로 올바르게 예측된 객체의 비율을 의미
# 'Recall' = 재현율
#  => 'Recall'은 실제 객체 중 모델이 올바르게 검출한 객체의 비율을 의미
# 'mAP@0.5' = Mean Average Precision (IoU=0.5에서의 mAP)
# 'mAP@0.5:0.95' = Mean Average Precision (IoU=0.5에서 0.95까지의 mAP)
#  => 'mAP'는 객체 검출 모델의 성능을 평가하는 대표적인 지표로,
#      해당 값을 통해 모델의 성능을 판별할 수 있음. (높을수록 학습이 잘 된 모델)

precision = metrics.box.p
recall = metrics.box.r
map_50 = metrics.box.map50
map_50_95 = metrics.box.map

print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"mAP@0.5: {map_50}")
print(f"mAP@0.5:0.95: {map_50_95}")

metrics.confusion_matrix.matrix
```

➔ 모델이 잘 학습되었는지를 평가하는 지표인 'Precision', 'Recall', 'mAP' 값을 출력

Unmanned & Intelligent systems CONtrol LAB

# 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [3 / 6]**

```
Ultralytics 8.4.35 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv12n summary (fused): 159 layers, 2,557,898 parameters, 0 gradients, 6.3 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2154.9±656.9 MB/s, size: 62.1 KB)
val: Scanning /content/roboflow/valid/labels.cache... 486 images, 1 backgrounds, 0 corrupt: 100% ──────
WARNING ⚠️ Box and segment counts should be equal, but got len(segments) = 82, len(boxes) = 1063. To re
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────
                   all        486       1063      0.686      0.714      0.751       0.56
                   BUS        105        139      0.624      0.763      0.725        0.6
                   CMT        151        192      0.682      0.672      0.722      0.597
            MOTORCYCLE        107        171      0.793      0.762      0.798      0.422
                   SED         94        212      0.657       0.66      0.699      0.511
                   SUV        136        185      0.584      0.611      0.692      0.536
                 TRUCK        132        164      0.775      0.817      0.869      0.696
Speed: 0.8ms preprocess, 2.9ms inference, 0.0ms loss, 2.3ms postprocess per image
Results saved to /content/runs/detect/val2
Precision: [    0.62449     0.68154     0.79313     0.65701      0.5836     0.77525]
Recall: [    0.76259     0.67188     0.76231     0.65959     0.61081     0.81707]
mAP@0.5: 0.7507414466121937
mAP@0.5:0.95: 0.5603171431103446
array([[       110,          0,          0,          1,          1,         14,         65],
       [         0,        130,          0,          6,         14,          1,         77],
       [         0,          0,        140,          0,          1,          0,         52],
       [         0,          2,          0,        144,          8,          1,        110],
       [         0,         16,          0,         13,        119,          0,         94],
       [         2,          7,          0,          1,          0,        127,         52],
       [        27,         37,         31,         47,         42,         21,          0]])
```

➔ Validation data를 기반으로 Validation을 수행한 뒤, 결과를 '/content/runs/detect/val2'에 저장

➔ 또한, 앞서 출력하였던 모델의 평가 지표인 'Precision', 'Recall', 'mAP' 값을 출력

평가 지표 설명 참고 → https://ctkim.tistory.com/entry/mAPMean-Average-Precision-%EC%A0%95%EB%A6%AC

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [4 / 6]**

```
import locale
locale.getpreferredencoding = lambda: "UTF-8"

!ls /content/runs/detect/val/
```

```
BoxF1_curve.png  confusion_matrix_normalized.png  val_batch1_labels.jpg
BoxP_curve.png   confusion_matrix.png             val_batch1_pred.jpg
BoxPR_curve.png  val_batch0_labels.jpg            val_batch2_labels.jpg
BoxR_curve.png   val_batch0_pred.jpg              val_batch2_pred.jpg
```

➔ Validation 결과가 저장되어 있는 경로상의 파일들 확인

**※ val을 실행할 때마다 결과 폴더가 val → val2 → val3 …으로 늘어납니다.** 출력 마지막 줄의 'Results saved to' 경로를 확인하세요. (46번 화면은 val2)

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [5 / 6]**

```
import locale
locale.getpreferredencoding = lambda: "UTF-8"

!pip install pillow

from PIL import Image
import matplotlib.pyplot as plt

# 이미지 파일 경로 설정 (예: 'example.jpg')
image_path = '/content/runs/detect/val2/val_batch2_labels.jpg'

# 이미지 열기
img = Image.open(image_path)

# 이미지 표시
plt.imshow(img)
plt.axis('off')  # 축 숨기기
plt.show()
```

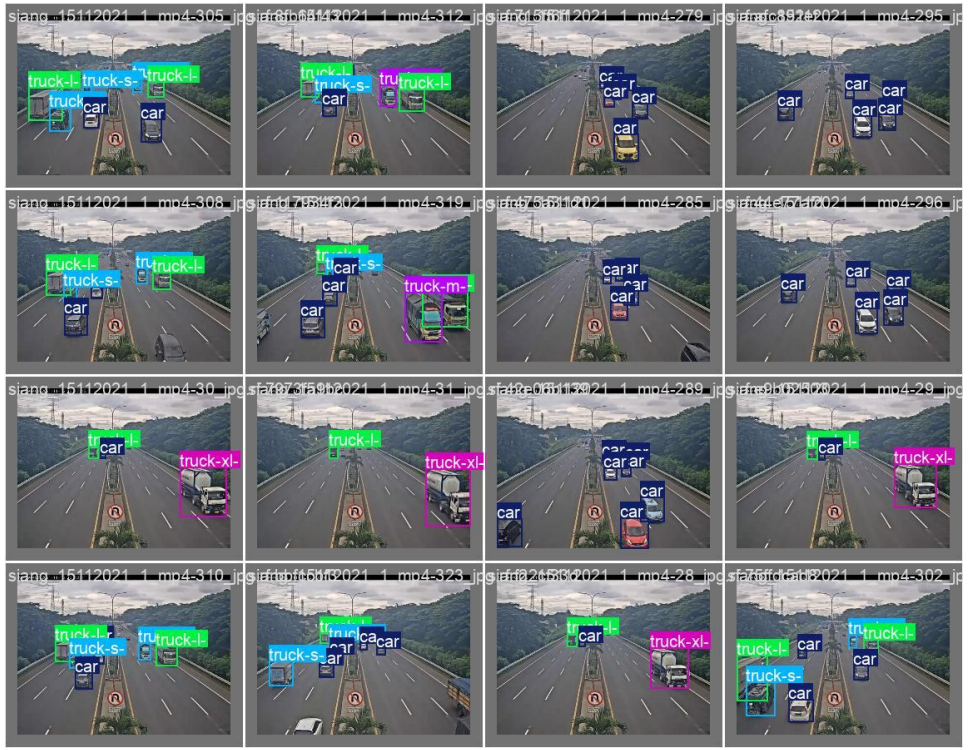
➔ Colab 환경에서 이미지를 화면에 출력하기 위해 'PIL(Python Imaging Library)'를 import 수행

➔ Validation 결과가 저장되어 있는 경로상의 파일 중, 본인이 원하는 파일의 경로를 작성

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 3) YOLO v12 기반 객체 인지 모델 검증

- **Google Colab 환경 내 YOLO v12 학습 모델 검증 [6 / 6]**



➔ 다음의 Validation 결과 이미지가 화면에 출력

➔ 인식한 객체를 Bounding box로 표시하며, 객체의 Class와 Confidence Score를 함께 출력

(Confidence Score란, 모델이 예측한 객체가 특정 Class에 속할 확률을 의미)

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 학습 · 검증 결과 파일 읽는 법

`results.png`

**epoch별 loss와 mAP 변화 그래프**

loss↓ mAP↑ 이면 순조로움 · val loss가 다시 오르면 과적합 신호

`confusion_matrix.png`

**실제 클래스 vs 예측 클래스 표**

대각선이 진할수록 좋음 · background 칸 = 오검출 · 미검출

`BoxPR_curve.png`

**클래스별 P-R 곡선**

오른쪽 위로 붙을수록 좋음 · 범례의 숫자 = 클래스별 AP

`BoxF1_curve.png`

**conf에 따른 F1 (P · R 조화평균)**

F1이 가장 높은 지점의 conf = 추천 임계값

`val_batch*_labels / _pred.jpg`

**같은 이미지의 정답 vs 예측**

놓친 물체와 오검출을 눈으로 확인

`weights/best.pt · last.pt`

**best: 검증 성능 최고 → 추론에 사용**

last: 마지막 epoch → 끊긴 학습 이어서 (resume)

학습 폴더: `/content/runs/exp` · 검증 폴더: `/content/runs/detect/val*`

UNICON
Unmanned & Intelligent systems CONtrol LAB

## 자주 만나는 오류와 해결 방법

| 증상 | 원인 | 해결 |
|---|---|---|
| **Dataset not found · data.yaml 오류** | data.yaml 경로가 Colab 경로와 다름 | train · val을 /content/... 절대경로로 수정 (36번) |
| **No labels found · Instances 0** | images ↔ labels 폴더 짝이 안 맞음 | 파일 이름이 같은지(.jpg ↔ .txt), 폴더명이 images · labels인지 확인 |
| **CUDA out of memory** | batch · imgsz가 GPU 메모리보다 큼 | batch=8 또는 batch=-1(자동), imgsz 줄이기 |
| **학습이 너무 느림** | GPU가 아니라 CPU로 실행 중 | 런타임 유형 → T4 GPU 확인 (28번), torch.cuda.is_available() |
| **Colab 연결 끊김 · GPU 한도** | 무료 GPU 사용 시간 제한 | 결과를 Drive에 저장 → YOLO('.../last.pt').train(resume=True) |
| **클래스 이름 이상 · IndexError** | data.yaml names와 라벨 번호 불일치 | names 개수 = 라벨 class 번호 최대값 + 1 (0부터 시작) |

UNICON
Unmanned & Intelligent systems CONtrol LAB

# 실 습

✓ Custom Dataset 기반의 YOLO v12 학습 수행 실습

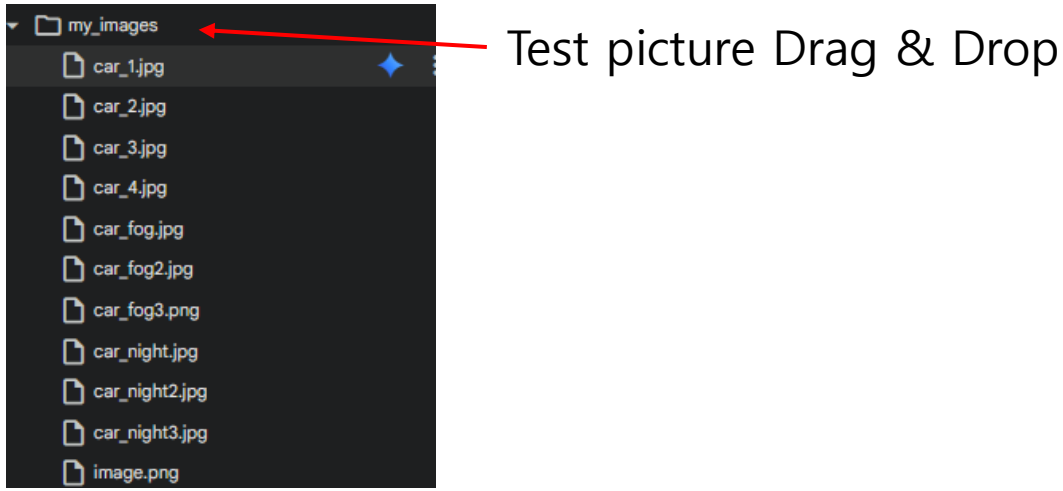
➔ **조건 1.** 30 epoch 이상 학습 수행하기

➔ **조건 2.** 학습한 "best.pt" 제외 사용 금지

## 폴더 생성

```
import os

os.makedirs('/content/my_images', exist_ok=True)
os.makedirs('/content/my_images_detected', exist_ok=True)
```




## 예제 코드

```
from ultralytics import YOLO

model = YOLO('/content/runs/exp/weights/best.pt')

model.predict(
    source='/content/my_images',
    conf=0.05, #detection 임계값
    save=True,
    project='/content',
    name='my_images_detected',
    exist_ok=True
)
```

※ conf=0.05는 매우 낮은 임계값 → 오검출이 많으면
0.25(기본값)로 올려 비교해 보세요

UNICON
Unmanned & Intelligent systems CONtrol LAB

# 실 습

## ✓ Custom Dataset 기반의 YOLO v12 학습 수행 실습

➔ **조건 1.** Roboflow에서 과일 관련 dataset 학습하기.

➔ **조건 2.** 30 epoch 이상 학습 수행하기

➔ **조건 3.** 1개의 Class가 아닌, 2개 이상의 Class를 바탕으로 학습 수행하기

실 습

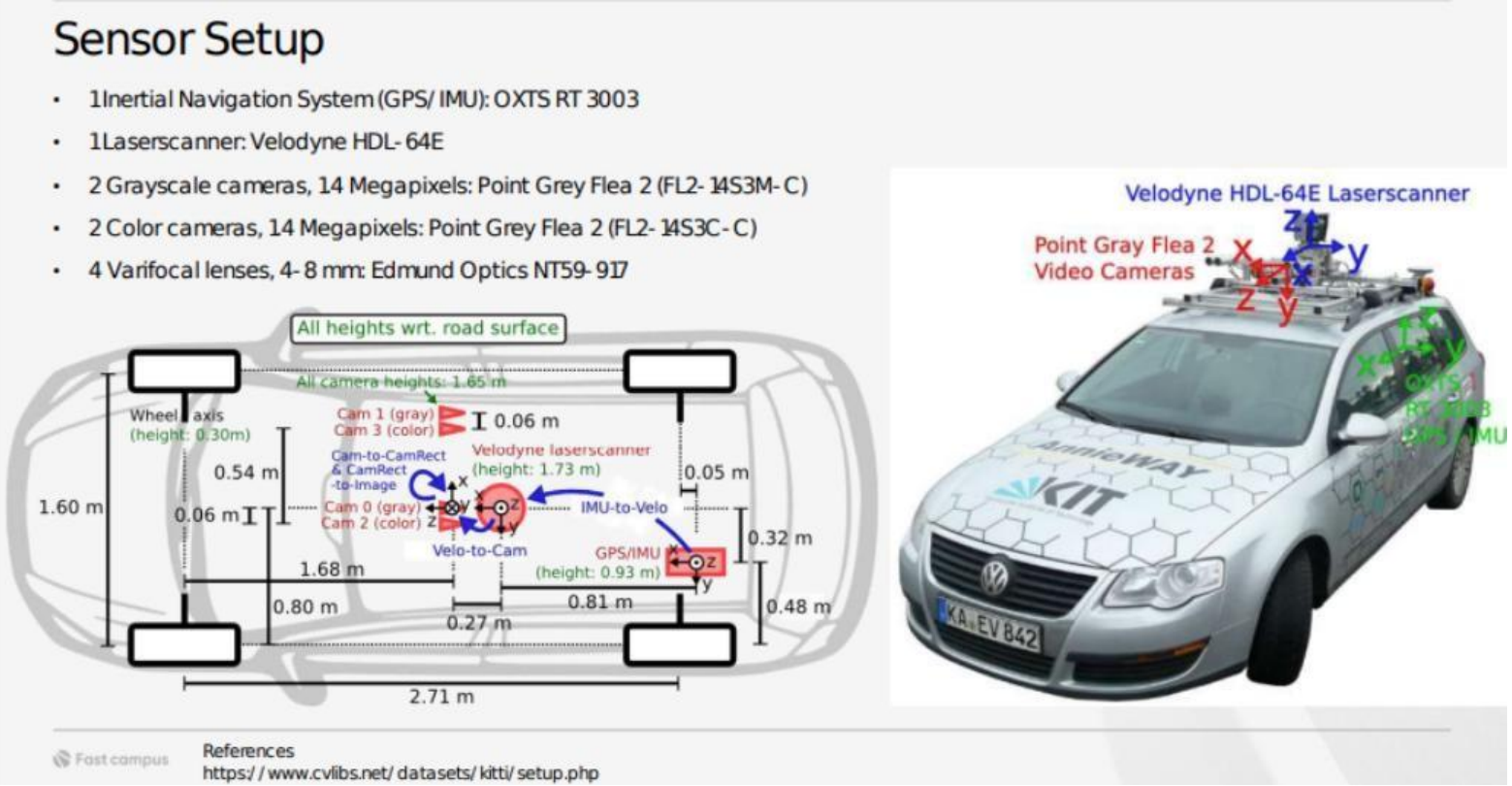
✓ Custom Dataset 기반의 YOLO v12 학습 수행 실습

➔ **조건 1.** Github의 fruit.mp4 파일 실시간 detection 하기.

➔ **조건 2.** 학습한 Best.pt 파일 사용하여 detection

## KITTI dataset 이란?

- LiDar와 여러 대의 Camera로 수집한 데이터와 이에 대한 라벨로 데이터셋 구성
- 제작업체 : KIT (Karlsruhe Institute of Technology)
- 데이터셋 공식 링크 : https://www.cvlibs.net/datasets/kitti/
- The KITTI Vision Benchmark Suite:
  https://www.youtube.com/watch?v=KXpZ6B1YB_k&t=10s



UNiCON
Unmanned & Intelligent systems CONtrol LAB

# KITTI dataset 이란?

- 3D Object Detection Dataset 링크:
- https://www.cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark=3d




데이터셋 구성

```
kitti
├── ImageSets
│   ├── train.txt
│   └── val.txt
├── training/
│   ├── 00/
│   │   ├── image_0
│   │   ├── image_1
│   │   ├── image_2
│   │   ├── image_3
│   │   ├── calib/
│   │   └── velodyne/
│   ...
│   └── 21
│       ├── image_0
│       ├── image_1
│       ├── image_2
│       ├── image_3
│       └── velodyne
└── testing/
    ├── 00/
    │   ├── image_0
    │   ├── image_1
    │   ├── image_2
    │   ├── image_3
    │   ├── calib/
    │   └── velodyne/
    ...
    └── 21
        ├── image_0
        ├── image_1
        ├── image_2
        ├── image_3
        └── velodyne
```

- Car
- Van
- Truck
- Pedestrians
- Sitters
- Cyclists
- Trams

UNICON
Unmanned & Intelligent systems CONtrol LAB

## KITTI dataset 이란?

- Dataset example



Image_00



Image_02



Image_01



Image_03

UNICON
Unmanned & Intelligent systems CONtrol LAB

# 실 습

✓ KITTI Dataset 기반의 YOLO v12 학습 수행 실습

➔ **조건 1.** 30 epoch 이상 학습 수행하기

➔ **조건 2.** 데이터셋의 라벨에 존재하는 Class를 탐지하는 모델 학습하기.

➔ **조건 3.** 1개의 Class가 아닌, 3개 이상의 Class를 바탕으로 학습 수행하기

TIP:

```
# KITTI 데이터셋 다운로드
!mkdir -p kitti_raw
%cd kitti_raw
!wget https://s3.eu-central-1.amazonaws.com/avg-kitti/data_object_image_2.zip
!wget https://s3.eu-central-1.amazonaws.com/avg-kitti/data_object_label_2.zip
!unzip -q data_object_image_2.zip
!unzip -q data_object_label_2.zip
%cd ..
```

UNICON
Unmanned & Intelligent systems CONtrol LAB

## Segmentation 이란?

**- Yolov11 Image Segmentation**

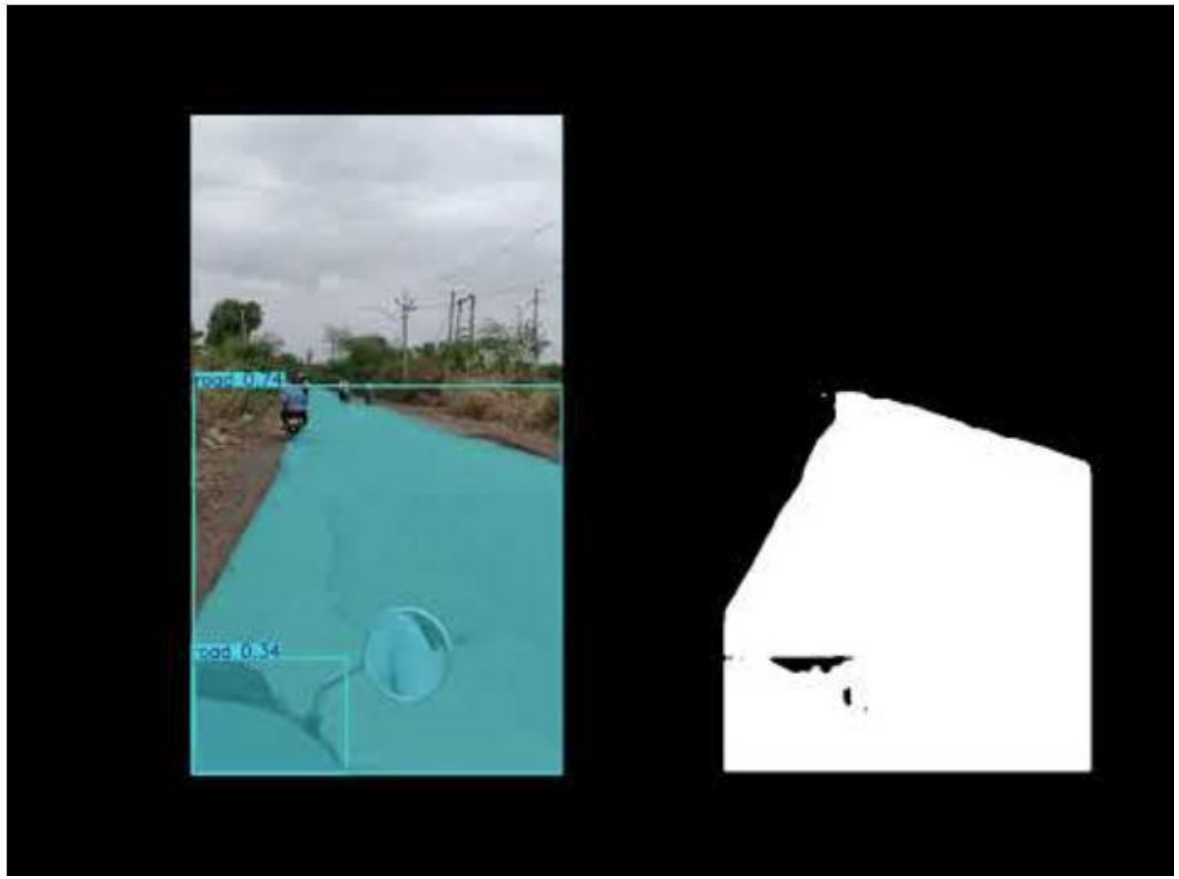



| 모델 | 크기 (픽셀) | mAP val 50-95 | 속도 CPU ONNX (ms) | 속도 T4 TensorRT10 (ms) | 파라미터 (M) | FLOPs (B) |
|---|---|---|---|---|---|---|
| YOLO11n | 640 | 39.5 | 56.1 ± 0.8 | 1.5 ± 0.0 | 2.6 | 6.5 |
| YOLO11s | 640 | 47.0 | 90.0 ± 1.2 | 2.5 ± 0.0 | 9.4 | 21.5 |
| YOLO11m | 640 | 51.5 | 183.2 ± 2.0 | 4.7 ± 0.1 | 20.1 | 68.0 |
| YOLO11l | 640 | 53.4 | 238.6 ± 1.4 | 6.2 ± 0.1 | 25.3 | 86.9 |
| YOLO11x | 640 | 54.7 | 462.8 ± 6.7 | 11.3 ± 0.2 | 56.9 | 194.9 |

UNICON
Unmanned & Intelligent systems CONtrol LAB

## Segmentation 이란?

- **Yolov12 Image Segmentation**
- **Road Seg example**



UNICON
Unmanned & Intelligent systems CONtrol LAB

# 실 습

## ✓ KITTI Dataset 기반의 YOLO v12 학습 수행 실습

: 기존의 객체 탐지(Object Detection)과는 달리, 본 실습은 **도로 형태를 마스크 형태로 예측하는 세그멘테이션 모델**을 목표로 합니다.

- ➔ **조건 1.** 30 epoch 이상 학습 수행하기
- ➔ **조건 2.** 도로를 세그멘테이션하는 모델 학습하기
- ➔ **조건 3.** 결과 시각화를 통해서 모델 평가해보기.

TIP:

- Roboflow의 데이터셋에는 KITTI데이터 기반 KITTI-Road-Segmentation 데이터 존재
- **Cityscapes, BDD100K등의 데이터셋도 존재함.**

## 실 습

✓ Real-time detection

➔ **조건 1.** Roboflow 데이터셋으로 best.pt 학습하여 사용

➔ **조건 2.** 데이터셋의 라벨에 존재하는 Class를 탐지하는 모델 학습하기.

➔ **조건 3.** 1개의 Class가 아닌, 3개 이상의 Class를 바탕으로 학습 수행하기

UNiCON
Unmanned & Intelligent systems CONtrol LAB

# QnA

E-mail : phj990608@hanyang.ac.kr

Mobile : 010-7282-0961